Parliamentary Inquiry Text Mining Report

Author: Juan Vargas T.

Email: [juan.vargastorres@student.uts.edu.au] (mailto:juan.vargastorres@student.uts.edu.au)

Website: jsebastianv.com

Repository: [github.com/Sebas102507/nlp-parliamentary-inquiry-text-mining] (https://github.com/Sebas102507/nlp-parliamentary-inquiry-text-mining)

Table of Contents

Use the links below to jump to each section.

- [0. Utilities](#0-utilities)
- [0.0 Environment setup](#00-environment-setup)
- [0.1 Imports and package setup](#01-imports-and-package-setup)
- [0.2 Global constants and visual theme](#02-global-constants-and-visual-theme)
- [0.3 Data models and metadata parsing classes](#03-data-models-and-metadata-parsing-classes)
- [0.4 Corpus ingestion service class](#04-corpus-ingestion-service-class)
- [0.5 Text preprocessing class and helper functions](#05-text-preprocessing-class-and-helper-functions)
- [1. Introduction](#1-introduction)
- [2. Data Ingestion](#2-data-ingestion)
- [3. Exploratory Data Analysis](#3-exploratory-data-analysis)
- [3.1 Corpus overview visuals](#31-corpus-overview-visuals)
- [3.2 Corpus composition by submitter type](#32-corpus-composition-by-submitter-type)
- [3.3 Distribution of document word counts](#33-distribution-of-document-word-counts)
- [3.4 Relationship between pages and word counts](#34-relationship-between-pages-and-word-counts)
- [3.5 Character counts by submitter type](#35-character-counts-by-submitter-type)
- [3.6 Linguistic annotation sample (NER and POS)](#36-linguistic-annotation-sample-ner-and-pos)
- [4. Preprocessing](#4-preprocessing)
- [4.1 Prepare cleaned tokens and lexical metrics](#41-prepare-cleaned-tokens-and-lexical-metrics)
- [4.2 Most common words](#42-most-common-words)
- [4.3 Corpus snapshot statistics](#43-corpus-snapshot-statistics)
- [4.4 Frequency table preparation (unigrams and NPMI bigrams)](#44-frequency-table-preparation-unigrams-npmi-bigrams-and-npmi-trigrams)
- [4.5 Top unigram frequency chart](#45-top-unigram-frequency-chart)
- [4.6 Bigram phrase visuals (frequency chart and network graph)](#46-bigram-phrase-visuals-frequency-chart-and-network-graph)
- [4.7 Top three-gram frequency chart](#47-top-three-gram-frequency-chart-using-npmi-phrase-output)
- [4.8 Word cloud of preprocessed corpus](#48-word-cloud-of-preprocessed-corpus)
- [5. Advanced Methods (Topic Modeling and Clustering)](#5-advanced-methods-topic-modeling-and-clustering)
- [5.1 Document vectorization from lemmatized tokens](#51-document-vectorization-

from-lemmatized-tokens)
- [5.2 LDA topic modeling with bag-of-words](#52-lda-topic-modeling-with-bag-of-words)
- [5.3 TF-IDF clustering experiment (hierarchical + K-means)](#53-tf-idf-clustering-experiment-hierarchical-k-means)
- [5.4 Word2Vec clustering experiment (hierarchical + K-means)](#54-word2vec-clustering-experiment-hierarchical-k-means)
- [6. Conclusion](#6-conclusion)
- [7. References](#7-references)

0. Utilities

This section centralizes reusable setup.

0.0 Environment setup

Run this once per environment to install all required libraries from the project requirements file.

In [1]:
```
# 0.0 Install required libraries
%pip install -r requirements.txt

import sys
import subprocess

if sys.version_info < (3, 14):
    subprocess.check_call([sys.executable, "-m", "spacy",
"download", "en_core_web_sm"])
else:
    print("spaCy model download skipped on Python 3.14+; NLTK
fallback will be used for NER.")
```

```
Requirement already satisfied: jupyter in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 1)) (1.1.1)
Requirement already satisfied: numpy in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 2)) (2.4.2)
Requirement already satisfied: pandas in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 3)) (3.0.1)
Requirement already satisfied: matplotlib in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 4)) (3.10.8)
Requirement already satisfied: seaborn in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 5)) (0.13.2)
Requirement already satisfied: nltk in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 6)) (3.9.3)
Requirement already satisfied: pymupdf in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 7)) (1.27.1)
Requirement already satisfied: scikit-learn in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 8)) (1.8.0)
Requirement already satisfied: wordcloud in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 9)) (1.9.6)
Requirement already satisfied: spacy in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 10)) (3.8.11)
Requirement already satisfied: gensim in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 11)) (4.4.0)
Requirement already satisfied: networkx in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 12)) (3.6.1)
Requirement already satisfied: umap-learn in
./.venv/lib/python3.12/site-packages (from -r requirements.txt
(line 13)) (0.5.11)
Requirement already satisfied: notebook in
```

```
./.venv/lib/python3.12/site-packages (from jupyter->-r
requirements.txt (line 1)) (7.5.4)
Requirement already satisfied: jupyter-console in
./.venv/lib/python3.12/site-packages (from jupyter->-r
requirements.txt (line 1)) (6.6.3)
Requirement already satisfied: nbconvert in
./.venv/lib/python3.12/site-packages (from jupyter->-r
requirements.txt (line 1)) (7.17.0)
Requirement already satisfied: ipykernel in
./.venv/lib/python3.12/site-packages (from jupyter->-r
requirements.txt (line 1)) (7.2.0)
Requirement already satisfied: ipywidgets in
./.venv/lib/python3.12/site-packages (from jupyter->-r
requirements.txt (line 1)) (8.1.8)
Requirement already satisfied: jupyterlab in
./.venv/lib/python3.12/site-packages (from jupyter->-r
requirements.txt (line 1)) (4.5.5)
Requirement already satisfied: python-dateutil>=2.8.2 in
./.venv/lib/python3.12/site-packages (from pandas->-r
requirements.txt (line 3)) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (26.0)
Requirement already satisfied: pillow>=8 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (12.1.1)
Requirement already satisfied: pyparsing>=3 in
./.venv/lib/python3.12/site-packages (from matplotlib->-r
requirements.txt (line 4)) (3.3.2)
Requirement already satisfied: click in
./.venv/lib/python3.12/site-packages (from nltk->-r
requirements.txt (line 6)) (8.3.1)
Requirement already satisfied: joblib in
./.venv/lib/python3.12/site-packages (from nltk->-r
requirements.txt (line 6)) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in
./.venv/lib/python3.12/site-packages (from nltk->-r
requirements.txt (line 6)) (2026.2.28)
Requirement already satisfied: tqdm in
./.venv/lib/python3.12/site-packages (from nltk->-r
requirements.txt (line 6)) (4.67.3)
```

```
Requirement already satisfied: scipy>=1.10.0 in
./.venv/lib/python3.12/site-packages (from scikit-learn->-r
requirements.txt (line 8)) (1.17.1)
Requirement already satisfied: threadpoolctl>=3.2.0 in
./.venv/lib/python3.12/site-packages (from scikit-learn->-r
requirements.txt (line 8)) (3.6.0)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (0.24.0)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (2.32.5)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,
<3.0.0,>=1.7.4 in ./.venv/lib/python3.12/site-packages (from
spacy->-r requirements.txt (line 10)) (2.12.5)
Requirement already satisfied: jinja2 in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (3.1.6)
Requirement already satisfied: setuptools in
./.venv/lib/python3.12/site-packages (from spacy->-r
requirements.txt (line 10)) (82.0.0)
Requirement already satisfied: annotated-types>=0.6.0 in
./.venv/lib/python3.12/site-packages (from
```

```
pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy->-r
requirements.txt (line 10)) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in
./.venv/lib/python3.12/site-packages (from
pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy->-r
requirements.txt (line 10)) (2.41.5)
Requirement already satisfied: typing-extensions>=4.14.1 in
./.venv/lib/python3.12/site-packages (from
pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy->-r
requirements.txt (line 10)) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in
./.venv/lib/python3.12/site-packages (from
pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy->-r
requirements.txt (line 10)) (0.4.2)
Requirement already satisfied: charset_normalizer<4,>=2 in
./.venv/lib/python3.12/site-packages (from
requests<3.0.0,>=2.13.0->spacy->-r requirements.txt (line 10))
(3.4.5)
Requirement already satisfied: idna<4,>=2.5 in
./.venv/lib/python3.12/site-packages (from
requests<3.0.0,>=2.13.0->spacy->-r requirements.txt (line 10))
(3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
./.venv/lib/python3.12/site-packages (from
requests<3.0.0,>=2.13.0->spacy->-r requirements.txt (line 10))
(2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
./.venv/lib/python3.12/site-packages (from
requests<3.0.0,>=2.13.0->spacy->-r requirements.txt (line 10))
(2026.2.25)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in
./.venv/lib/python3.12/site-packages (from thinc<8.4.0,>=8.3.4-
>spacy->-r requirements.txt (line 10)) (1.3.3)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in
./.venv/lib/python3.12/site-packages (from thinc<8.4.0,>=8.3.4-
>spacy->-r requirements.txt (line 10)) (0.1.5)
Requirement already satisfied: typer>=0.24.0 in
./.venv/lib/python3.12/site-packages (from typer-
slim<1.0.0,>=0.3.0->spacy->-r requirements.txt (line 10))
(0.24.1)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in
./.venv/lib/python3.12/site-packages (from
weasel<0.5.0,>=0.4.2->spacy->-r requirements.txt (line 10))
(0.23.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in
./.venv/lib/python3.12/site-packages (from
weasel<0.5.0,>=0.4.2->spacy->-r requirements.txt (line 10))
(7.5.1)
Requirement already satisfied: wrapt in
./.venv/lib/python3.12/site-packages (from smart-
open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2->spacy->-r
requirements.txt (line 10)) (2.1.2)
Requirement already satisfied: numba>=0.51.2 in
```

```
./.venv/lib/python3.12/site-packages (from umap-learn->-r
requirements.txt (line 13)) (0.64.0)
Requirement already satisfied: pynndescent>=0.5 in
./.venv/lib/python3.12/site-packages (from umap-learn->-r
requirements.txt (line 13)) (0.6.0)
Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in
./.venv/lib/python3.12/site-packages (from numba>=0.51.2->umap-
learn->-r requirements.txt (line 13)) (0.46.0)
Requirement already satisfied: six>=1.5 in
./.venv/lib/python3.12/site-packages (from python-
dateutil>=2.8.2->pandas->-r requirements.txt (line 3)) (1.17.0)
Requirement already satisfied: shellingham>=1.3.0 in
./.venv/lib/python3.12/site-packages (from typer>=0.24.0-
>typer-slim<1.0.0,>=0.3.0->spacy->-r requirements.txt (line
10)) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in
./.venv/lib/python3.12/site-packages (from typer>=0.24.0-
>typer-slim<1.0.0,>=0.3.0->spacy->-r requirements.txt (line
10)) (14.3.3)
Requirement already satisfied: annotated-doc>=0.0.2 in
./.venv/lib/python3.12/site-packages (from typer>=0.24.0-
>typer-slim<1.0.0,>=0.3.0->spacy->-r requirements.txt (line
10)) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
./.venv/lib/python3.12/site-packages (from rich>=12.3.0-
>typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy->-r
requirements.txt (line 10)) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
./.venv/lib/python3.12/site-packages (from rich>=12.3.0-
>typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy->-r
requirements.txt (line 10)) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in
./.venv/lib/python3.12/site-packages (from markdown-it-
py>=2.2.0->rich>=12.3.0->typer>=0.24.0->typer-
slim<1.0.0,>=0.3.0->spacy->-r requirements.txt (line 10))
(0.1.2)
Requirement already satisfied: appnope>=0.1.2 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (0.1.4)
Requirement already satisfied: comm>=0.1.1 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (0.2.3)
Requirement already satisfied: debugpy>=1.6.5 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (1.8.20)
Requirement already satisfied: ipython>=7.23.1 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (9.11.0)
Requirement already satisfied: jupyter-client>=8.8.0 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (8.8.0)
Requirement already satisfied: jupyter-core!=6.0.*,>=5.1 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
```

```
>-r requirements.txt (line 1)) (5.9.1)
Requirement already satisfied: matplotlib-inline>=0.1 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (0.2.1)
Requirement already satisfied: nest-asyncio>=1.4 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (1.6.0)
Requirement already satisfied: psutil>=5.7 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (7.2.2)
Requirement already satisfied: pyzmq>=25 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (27.1.0)
Requirement already satisfied: tornado>=6.4.1 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (6.5.4)
Requirement already satisfied: traitlets>=5.4.0 in
./.venv/lib/python3.12/site-packages (from ipykernel->jupyter-
>-r requirements.txt (line 1)) (5.14.3)
Requirement already satisfied: decorator>=5.1.0 in
./.venv/lib/python3.12/site-packages (from ipython>=7.23.1-
>ipykernel->jupyter->-r requirements.txt (line 1)) (5.2.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0
in ./.venv/lib/python3.12/site-packages (from ipython>=7.23.1-
>ipykernel->jupyter->-r requirements.txt (line 1)) (1.1.1)
Requirement already satisfied: jedi>=0.18.2 in
./.venv/lib/python3.12/site-packages (from ipython>=7.23.1-
>ipykernel->jupyter->-r requirements.txt (line 1)) (0.19.2)
Requirement already satisfied: pexpect>4.6 in
./.venv/lib/python3.12/site-packages (from ipython>=7.23.1-
>ipykernel->jupyter->-r requirements.txt (line 1)) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
./.venv/lib/python3.12/site-packages (from ipython>=7.23.1-
>ipykernel->jupyter->-r requirements.txt (line 1)) (3.0.52)
Requirement already satisfied: stack_data>=0.6.0 in
./.venv/lib/python3.12/site-packages (from ipython>=7.23.1-
>ipykernel->jupyter->-r requirements.txt (line 1)) (0.6.3)
Requirement already satisfied: wcwidth in
./.venv/lib/python3.12/site-packages (from
prompt_toolkit<3.1.0,>=3.0.41->ipython>=7.23.1->ipykernel-
>jupyter->-r requirements.txt (line 1)) (0.6.0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
./.venv/lib/python3.12/site-packages (from jedi>=0.18.2-
>ipython>=7.23.1->ipykernel->jupyter->-r requirements.txt (line
1)) (0.8.6)
Requirement already satisfied: platformdirs>=2.5 in
./.venv/lib/python3.12/site-packages (from jupyter-
core!=6.0.*,>=5.1->ipykernel->jupyter->-r requirements.txt
(line 1)) (4.9.4)
Requirement already satisfied: ptyprocess>=0.5 in
./.venv/lib/python3.12/site-packages (from pexpect>4.6-
>ipython>=7.23.1->ipykernel->jupyter->-r requirements.txt (line
1)) (0.7.0)
```

```
Requirement already satisfied: executing>=1.2.0 in
./.venv/lib/python3.12/site-packages (from stack_data>=0.6.0-
>ipython>=7.23.1->ipykernel->jupyter->-r requirements.txt (line
1)) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in
./.venv/lib/python3.12/site-packages (from stack_data>=0.6.0-
>ipython>=7.23.1->ipykernel->jupyter->-r requirements.txt (line
1)) (3.0.1)
Requirement already satisfied: pure-eval in
./.venv/lib/python3.12/site-packages (from stack_data>=0.6.0-
>ipython>=7.23.1->ipykernel->jupyter->-r requirements.txt (line
1)) (0.2.3)
Requirement already satisfied: widgetsnbextension~=4.0.14 in
./.venv/lib/python3.12/site-packages (from ipywidgets->jupyter-
>-r requirements.txt (line 1)) (4.0.15)
Requirement already satisfied: jupyterlab_widgets~=3.0.15 in
./.venv/lib/python3.12/site-packages (from ipywidgets->jupyter-
>-r requirements.txt (line 1)) (3.0.16)
Requirement already satisfied: MarkupSafe>=2.0 in
./.venv/lib/python3.12/site-packages (from jinja2->spacy->-r
requirements.txt (line 10)) (3.0.3)
Requirement already satisfied: async-lru>=1.0.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab->jupyter-
>-r requirements.txt (line 1)) (2.2.0)
Requirement already satisfied: httpx<1,>=0.25.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab->jupyter-
>-r requirements.txt (line 1)) (0.28.1)
Requirement already satisfied: jupyter-lsp>=2.0.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab->jupyter-
>-r requirements.txt (line 1)) (2.3.0)
Requirement already satisfied: jupyter-server<3,>=2.4.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab->jupyter-
>-r requirements.txt (line 1)) (2.17.0)
Requirement already satisfied: jupyterlab-server<3,>=2.28.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab->jupyter-
>-r requirements.txt (line 1)) (2.28.0)
Requirement already satisfied: notebook-shim>=0.2 in
./.venv/lib/python3.12/site-packages (from jupyterlab->jupyter-
>-r requirements.txt (line 1)) (0.2.4)
Requirement already satisfied: anyio in
./.venv/lib/python3.12/site-packages (from httpx<1,>=0.25.0-
>jupyterlab->jupyter->-r requirements.txt (line 1)) (4.12.1)
Requirement already satisfied: httpcore==1.* in
./.venv/lib/python3.12/site-packages (from httpx<1,>=0.25.0-
>jupyterlab->jupyter->-r requirements.txt (line 1)) (1.0.9)
Requirement already satisfied: h11>=0.16 in
./.venv/lib/python3.12/site-packages (from httpcore==1.*-
>httpx<1,>=0.25.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (0.16.0)
Requirement already satisfied: argon2-cffi>=21.1 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (25.1.0)
```

```
Requirement already satisfied: jupyter-events>=0.11.0 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (0.12.0)
Requirement already satisfied: jupyter-server-terminals>=0.4.4
in ./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (0.5.4)
Requirement already satisfied: nbformat>=5.3.0 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (5.10.4)
Requirement already satisfied: prometheus-client>=0.9 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (0.24.1)
Requirement already satisfied: send2trash>=1.8.2 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (2.1.0)
Requirement already satisfied: terminado>=0.8.3 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (0.18.1)
Requirement already satisfied: websocket-client>=1.7 in
./.venv/lib/python3.12/site-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (1.9.0)
Requirement already satisfied: babel>=2.10 in
./.venv/lib/python3.12/site-packages (from jupyterlab-
server<3,>=2.28.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (2.18.0)
Requirement already satisfied: json5>=0.9.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab-
server<3,>=2.28.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (0.13.0)
Requirement already satisfied: jsonschema>=4.18.0 in
./.venv/lib/python3.12/site-packages (from jupyterlab-
server<3,>=2.28.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (4.26.0)
Requirement already satisfied: argon2-cffi-bindings in
./.venv/lib/python3.12/site-packages (from argon2-cffi>=21.1-
>jupyter-server<3,>=2.4.0->jupyterlab->jupyter->-r
requirements.txt (line 1)) (25.1.0)
Requirement already satisfied: attrs>=22.2.0 in
./.venv/lib/python3.12/site-packages (from jsonschema>=4.18.0-
>jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter->-r
requirements.txt (line 1)) (25.4.0)
Requirement already satisfied: jsonschema-
specifications>=2023.03.6 in ./.venv/lib/python3.12/site-
packages (from jsonschema>=4.18.0->jupyterlab-
server<3,>=2.28.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (2025.9.1)
```

```
Requirement already satisfied: referencing>=0.28.4 in
./.venv/lib/python3.12/site-packages (from jsonschema>=4.18.0-
>jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter->-r
requirements.txt (line 1)) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in
./.venv/lib/python3.12/site-packages (from jsonschema>=4.18.0-
>jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter->-r
requirements.txt (line 1)) (0.30.0)
Requirement already satisfied: python-json-logger>=2.0.4 in
./.venv/lib/python3.12/site-packages (from jupyter-
events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter-
>-r requirements.txt (line 1)) (4.0.0)
Requirement already satisfied: pyyaml>=5.3 in
./.venv/lib/python3.12/site-packages (from jupyter-
events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter-
>-r requirements.txt (line 1)) (6.0.3)
Requirement already satisfied: rfc3339-validator in
./.venv/lib/python3.12/site-packages (from jupyter-
events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter-
>-r requirements.txt (line 1)) (0.1.4)
Requirement already satisfied: rfc3986-validator>=0.1.1 in
./.venv/lib/python3.12/site-packages (from jupyter-
events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter-
>-r requirements.txt (line 1)) (0.1.1)
Requirement already satisfied: fqdn in
./.venv/lib/python3.12/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (1.5.1)
Requirement already satisfied: isoduration in
./.venv/lib/python3.12/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (20.11.0)
Requirement already satisfied: jsonpointer>1.13 in
./.venv/lib/python3.12/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (3.0.0)
Requirement already satisfied: rfc3987-syntax>=1.1.0 in
./.venv/lib/python3.12/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (1.1.0)
Requirement already satisfied: uri-template in
./.venv/lib/python3.12/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
(line 1)) (1.3.0)
Requirement already satisfied: webcolors>=24.6.0 in
./.venv/lib/python3.12/site-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter->-r requirements.txt
```

```
(line 1)) (25.10.0)
Requirement already satisfied: beautifulsoup4 in
./.venv/lib/python3.12/site-packages (from nbconvert->jupyter-
>-r requirements.txt (line 1)) (4.14.3)
Requirement already satisfied: bleach!=5.0.0 in
./.venv/lib/python3.12/site-packages (from bleach[css]!=5.0.0-
>nbconvert->jupyter->-r requirements.txt (line 1)) (6.3.0)
Requirement already satisfied: defusedxml in
./.venv/lib/python3.12/site-packages (from nbconvert->jupyter-
>-r requirements.txt (line 1)) (0.7.1)
Requirement already satisfied: jupyterlab-pygments in
./.venv/lib/python3.12/site-packages (from nbconvert->jupyter-
>-r requirements.txt (line 1)) (0.3.0)
Requirement already satisfied: mistune<4,>=2.0.3 in
./.venv/lib/python3.12/site-packages (from nbconvert->jupyter-
>-r requirements.txt (line 1)) (3.2.0)
Requirement already satisfied: nbclient>=0.5.0 in
./.venv/lib/python3.12/site-packages (from nbconvert->jupyter-
>-r requirements.txt (line 1)) (0.10.4)
Requirement already satisfied: pandocfilters>=1.4.1 in
./.venv/lib/python3.12/site-packages (from nbconvert->jupyter-
>-r requirements.txt (line 1)) (1.5.1)
Requirement already satisfied: webencodings in
./.venv/lib/python3.12/site-packages (from bleach!=5.0.0-
>bleach[css]!=5.0.0->nbconvert->jupyter->-r requirements.txt
(line 1)) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in
./.venv/lib/python3.12/site-packages (from bleach[css]!=5.0.0-
>nbconvert->jupyter->-r requirements.txt (line 1)) (1.4.0)
Requirement already satisfied: fastjsonschema>=2.15 in
./.venv/lib/python3.12/site-packages (from nbformat>=5.3.0-
>jupyter-server<3,>=2.4.0->jupyterlab->jupyter->-r
requirements.txt (line 1)) (2.21.2)
Requirement already satisfied: lark>=1.2.2 in
./.venv/lib/python3.12/site-packages (from rfc3987-
syntax>=1.1.0->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter-
>-r requirements.txt (line 1)) (1.3.1)
Requirement already satisfied: cffi>=1.0.1 in
./.venv/lib/python3.12/site-packages (from argon2-cffi-
bindings->argon2-cffi>=21.1->jupyter-server<3,>=2.4.0-
>jupyterlab->jupyter->-r requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: pycparser in
./.venv/lib/python3.12/site-packages (from cffi>=1.0.1->argon2-
cffi-bindings->argon2-cffi>=21.1->jupyter-server<3,>=2.4.0-
>jupyterlab->jupyter->-r requirements.txt (line 1)) (3.0)
Requirement already satisfied: soupsieve>=1.6.1 in
./.venv/lib/python3.12/site-packages (from beautifulsoup4-
>nbconvert->jupyter->-r requirements.txt (line 1)) (2.8.3)
Requirement already satisfied: arrow>=0.15.0 in
./.venv/lib/python3.12/site-packages (from isoduration-
>jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0-
>jupyter-server<3,>=2.4.0->jupyterlab->jupyter->-r
```

```
requirements.txt (line 1)) (1.4.0)
Requirement already satisfied: tzdata in
./.venv/lib/python3.12/site-packages (from arrow>=0.15.0-
>isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.11.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter-
>-r requirements.txt (line 1)) (2025.3)
Note: you may need to restart the kernel to use updated
packages.
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-
models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-
3.8.0-py3-none-any.whl (12.8 MB)
 [2K     [90m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ [0m
 [32m12.8/12.8 MB [0m  [31m12.7 MB/s [0m   [33m0:00:01 [0m eta
 [36m0:00:01 [0m
 [?25h [38;5;2m✔ Download and installation successful [0m
You can now load the package via spacy.load('en_core_web_sm')
```

0.1 Imports and package setup

Load required Python libraries and NLTK resources used throughout the notebook.

In [2]:
```
from pathlib import Path
import os
import re
import string
import logging
import sys
import subprocess
from dataclasses import dataclass
from collections import Counter
from typing import Iterable

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import LinearSegmentedColormap, Normalize
from mpl_toolkits.axes_grid1 import make_axes_locatable
from scipy.cluster.hierarchy import dendrogram, linkage
from wordcloud import WordCloud
import textwrap

try:
    import fitz
except ModuleNotFoundError:
    subprocess.check_call([sys.executable, "-m", "pip", "install",
"-q", "pymupdf"])
    import fitz

try:
    from gensim.corpora import Dictionary
    from gensim.models.coherencemodel import CoherenceModel
    from gensim.models.phrases import Phrases, Phraser
except ModuleNotFoundError:
    subprocess.check_call([sys.executable, "-m", "pip", "install",
"-q", "gensim"])
    from gensim.corpora import Dictionary
    from gensim.models.coherencemodel import CoherenceModel
    from gensim.models.phrases import Phrases, Phraser

try:
    import networkx as nx
except ModuleNotFoundError:
    subprocess.check_call([sys.executable, "-m", "pip", "install",
"-q", "networkx"])
    import networkx as nx

import nltk
from nltk.corpus import stopwords, wordnet as wn
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag, ne_chunk, trigrams
```

```
from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.decomposition import LatentDirichletAllocation,
TruncatedSVD
from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.metrics import silhouette_samples, silhouette_score

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)
nltk.download("averaged_perceptron_tagger_eng", quiet=True)
nltk.download("maxent_ne_chunker_tab", quiet=True)
nltk.download("words", quiet=True)

LOGGER = logging.getLogger()
LOGGER.setLevel(logging.INFO)
```

0.2 Global constants and visual theme

Define palette values, stopword settings, label mappings, and plotting defaults.

In [3]:
```
UTS_PALETTE = {
    "blue": "#0F4BEB",
    "red": "#FF2305",
    "black": "#000000",
    "dark_grey": "#323232",
    "mid_grey": "#B2B2B2",
    "light_grey": "#EBEBEB",
    "white": "#FFFFFF",
}

SUBMISSIONS_DIR = Path("docs/submissions")
HANSARD_DIR = Path("docs/hansard_transcripts")
DOMAIN_STOPWORDS = {
    "submission", "submissions", "inquiry", "committee",
"electoral",
    "election", "federal", "australian", "australia", "parliament"
}

PUNCT_SET = set(string.punctuation).union({
    "...", "''", "``", "’", "“", "”", "–", "—", ".", ","
})

SPACY_ENTITY_LABEL_DESCRIPTIONS = {
    "PERSON": "People, including fictional",
    "NORP": "Nationalities, Religious, or Political groups",
    "FAC": "Buildings, Airports, Highways, Bridges, etc.",
    "ORG": "Companies, Agencies, Institutions, etc.",
    "GPE": "Countries, Cities, States",
    "LOC": "Non-GPE locations, Mountain ranges, Bodies of water",
    "PRODUCT": "Objects, Vehicles, Foods, etc. (Not services)",
    "EVENT": "Named hurricanes, battles, wars, sports events, etc.",
    "WORK_OF_ART": "Titles of books, songs, etc.",
    "LAW": "Named documents made into laws",
    "LANGUAGE": "Any named language",
    "DATE": "Absolute or relative dates or periods",
    "TIME": "Times smaller than a day",
    "PERCENT": "Percentage",
    "MONEY": "Monetary values, including unit",
    "QUANTITY": "Measurements, as of weight or distance",
    "ORDINAL": "First, second, etc.",
    "CARDINAL": "Numerals that do not fall under another type",
}

ORG_NAME_INDICATORS = {
    "association", "assn", "institute", "foundation", "committee",
"commission",
    "council", "union", "party", "group", "alliance", "network",
"society",
    "organisation", "organization", "org", "company", "co", "corp",
"corporation",
    "inc", "inc.", "ltd", "ltd.", "limited", "pty", "pty.", "trust",
"church",
    "club", "board", "authority", "agency", "department", "office",
```

```
"branch",
}
ORG_OVERRIDE_LABELS = {"NORP", "CARDINAL", "GPE"}

PENN_POS_TAG_MAP = {
    "CC": "Coordinating conjunction",
    "CD": "Cardinal number",
    "DT": "Determiner",
    "EX": "Existential there",
    "FW": "Foreign word",
    "IN": "Preposition/subordinating conjunction",
    "JJ": "Adjective",
    "JJR": "Adjective, comparative",
    "JJS": "Adjective, superlative",
    "LS": "List item marker",
    "MD": "Modal",
    "NN": "Noun, singular/common",
    "NNS": "Noun, plural",
    "NNP": "Proper noun, singular",
    "NNPS": "Proper noun, plural",
    "PDT": "Predeterminer",
    "POS": "Possessive ending",
    "PRP": "Personal pronoun",
    "PRP$": "Possessive pronoun",
    "RB": "Adverb",
    "RBR": "Adverb, comparative",
    "RBS": "Adverb, superlative",
    "RP": "Particle",
    "TO": "to",
    "UH": "Interjection",
    "VB": "Verb, base form",
    "VBD": "Verb, past tense",
    "VBG": "Verb, gerund/present participle",
    "VBN": "Verb, past participle",
    "VBP": "Verb, non-3rd person singular present",
    "VBZ": "Verb, 3rd person singular present",
    "WDT": "Wh-determiner",
    "WP": "Wh-pronoun",
    "WP$": "Possessive wh-pronoun",
    "WRB": "Wh-adverb",
}

sns.set_theme(style="whitegrid")
plt.rcParams["figure.facecolor"] = UTS_PALETTE["white"]
plt.rcParams["axes.facecolor"] = UTS_PALETTE["white"]
plt.rcParams["axes.edgecolor"] = UTS_PALETTE["dark_grey"]
plt.rcParams["axes.labelcolor"] = UTS_PALETTE["dark_grey"]
plt.rcParams["text.color"] = UTS_PALETTE["dark_grey"]

NLP = None
try:
    import spacy
    NLP = spacy.load("en_core_web_sm")
```

```
    LOGGER.debug("Spacy model successfuly loaded")

except Exception:
    LOGGER.warning("spaCy is unavailable in this Python environment;
NLTK fallback will be used for NER.")
```

`DOMAIN_STOPWORDS` contains words that are contextually frequent but not meaningful for semantic analysis
in the parliamentary submissions corpus. These include words describing the process or context
("submission", "submissions", "inquiry", "committee", "electoral", "election", "federal", "australian", "australia", "parliament") that do not add value.

`PUNCT_SET` includes standard punctuation plus additional typographical marks and quotation variants
that may occur in parliamentary submissions. This is to robustly detect and remove all forms of
punctuation, including ellipses and different quote and dash styles, for thorough token cleaning.

0.3 Data models and metadata parsing classes

Define configuration/data classes and metadata parsing logic.

In [4]:

```
@dataclass(frozen=True)
class CorpusConfig:
    """Configuration object for ingestion and basic corpus
filtering."""
    docs_dir: Path
    min_word_count: int = 1
```

In [5]:
```
class MetadataParser:
    """Parses metadata and provides metadata linked NER
utilities."""

    @staticmethod
    def infer_submitter_type(name: str, nlp_model=None) -> str:
        """Infer submitter category from submitter name using spaCy
NER labels."""
        lowered = name.lower()
        if "withheld" in lowered or "name withheld" in lowered or
"withhold" in lowered:
            return "Withheld"

        model = nlp_model if nlp_model is not None else NLP
        if model is None:
            return "Unknown"

        doc = model(name)
        labels = [ent.label_ for ent in doc.ents if ent.label_]
        unique_labels = list(dict.fromkeys(labels))
        filtered_labels = [label for label in unique_labels if label
!= "DATE"]

        if not filtered_labels:
            # Fallback heuristic: infer organisation when common org
tokens appear.
            if any(indicator in lowered for indicator in
ORG_NAME_INDICATORS):
                return SPACY_ENTITY_LABEL_DESCRIPTIONS["ORG"]
            return "Unknown"

        # Keep a single label with priority: ORG > PERSON > first
remaining.
        if "ORG" in filtered_labels:
            selected_label = "ORG"
        elif "PERSON" in filtered_labels:
            selected_label = "PERSON"
        else:
            selected_label = filtered_labels[0]

        if selected_label in ORG_OVERRIDE_LABELS:
            selected_label = "ORG"

        return SPACY_ENTITY_LABEL_DESCRIPTIONS.get(selected_label,
selected_label)

    @staticmethod
    def parse_filename(file_name: str) -> tuple[str, str, str]:
        """Parse file name into submission id, submitter name, and
submitter type."""
        base = file_name.replace(".pdf", "")
        id_match = re.match(r"^(SupSub\d+\.\d+|Sub\d+
(?:\sAtt\d+)?)", base)
```

```
        submission_id = id_match.group(1) if id_match else "Unknown"

        parts = base.split(" - ", 1)
        submitter = parts[1].strip() if len(parts) > 1 else
"Unknown"
        submitter_type =
MetadataParser.infer_submitter_type(submitter)
        return submission_id, submitter, submitter_type

    @staticmethod
    def parse_hansard_filename(file_name: str) -> tuple[str, str,
str]:
        """Parse Hansard transcript filename into hearing ID,
committee name, and submitter type."""
        base = file_name.replace(".pdf", "")
        date_match = re.search(r"(\d{4}_\d{2}_\d{2})$", base)
        if date_match:
            hearing_date = date_match.group(1).replace("_", "-")
            hearing_id = f"Hansard_{hearing_date}"
            committee_name = base[: date_match.start()].rstrip("_")
        else:
            hearing_id = base
            committee_name = base
        submitter_type = "Hansard"
        return hearing_id, committee_name, submitter_type

    @staticmethod
    def extract_named_entities(text_series: pd.Series,
nlp_model=None, max_docs: int | None = 120) -> pd.DataFrame:
        """Extract named entities using batched spaCy processing or
NLTK fallback."""
        records = []
        iterable = text_series.head(max_docs) if max_docs is not
None else text_series

        if nlp_model is not None:
            texts = iterable.fillna("").astype(str).tolist()
            LOGGER.debug("NER extraction started with spaCy batching
(max_docs=%s)", max_docs)

            try:
                doc_stream = nlp_model.pipe(texts, batch_size=20,
n_process=-1)
                for doc_id, doc in enumerate(doc_stream):
                    for ent in doc.ents:
                        if ent.text.strip():
                            records.append(
                                {
                                    "doc_id": doc_id,
                                    "entity": ent.text.strip(),
                                    "label": ent.label_,
                                }
                            )
```

```
            except Exception as exc:
                LOGGER.warning(
                    "Parallel spaCy NER failed (%s); retrying with
single-process batching.",
                    exc,
                )
                for doc_id, doc in enumerate(nlp_model.pipe(texts,
batch_size=20)):
                    for ent in doc.ents:
                        if ent.text.strip():
                            records.append(
                                {
                                    "doc_id": doc_id,
                                    "entity": ent.text.strip(),
                                    "label": ent.label_,
                                }
                            )

            LOGGER.debug("NER extraction finished with spaCy: %d
entities", len(records))
            return pd.DataFrame(records)

        LOGGER.debug("spaCy unavailable; using NLTK fallback for NER
(max_docs=%s)", max_docs)
        for doc_id, text in enumerate(iterable):
            tokens = word_tokenize(str(text))
            if not tokens:
                continue

            tagged = pos_tag(tokens)
            chunks = ne_chunk(tagged, binary=False)

            for chunk in chunks:
                if hasattr(chunk, "label"):
                    entity_text = " ".join(token for token, _ in
chunk.leaves()).strip()
                    if entity_text:
                        records.append(
                            {
                                "doc_id": doc_id,
                                "entity": entity_text,
                                "label": chunk.label(),
                            }
                        )

        return pd.DataFrame(records)
```

0.4 Corpus ingestion service class

Define the service that reads PDFs and builds ingestion records.

In [6]:
```
# 0.4 Corpus ingestion service class

class CorpusIngestionService:
    """Single responsibility service for PDF extraction and corpus
construction."""

    def __init__(self, config: CorpusConfig):
        """Initialize service with runtime corpus configuration."""
        self.config = config

    def extract_pdf_text(self, pdf_path: Path) -> tuple[str, int,
bool]:
        """Extract text from a PDF and return text, page count, and
extraction status."""
        text_parts = []
        page_count = 0
        extraction_ok = True

        try:
            devnull_fd = os.open(os.devnull, os.O_WRONLY)
            old_stderr_fd = os.dup(2)
            os.dup2(devnull_fd, 2)
            os.close(devnull_fd)
            try:
                with fitz.open(str(pdf_path)) as doc:
                    page_count = len(doc)
                    for page in doc:
                        text_parts.append(page.get_text("text") or
"")
            finally:
                os.dup2(old_stderr_fd, 2)
                os.close(old_stderr_fd)
        except Exception as exc:
            extraction_ok = False
            LOGGER.warning("Failed to extract PDF: %s | error=%s",
pdf_path.name, exc)

        raw_text = "\n".join(text_parts).strip()
        return raw_text, page_count, extraction_ok

    def read_corpus(self, evidence_style: str = "Submission") ->
pd.DataFrame:
        """Read all PDFs from the configured directory and build a
filtered corpus dataframe.

        Parameters
        ----------
        evidence_style : str
            Label for the evidence type column -- ``"Submission"``
for written
            evidence or ``"Hansard"`` for oral hearing transcripts.
        """
        pdf_paths = sorted(self.config.docs_dir.glob("*.pdf"))
```

```
        if not pdf_paths:
            raise FileNotFoundError(
                f"No PDFs found in {self.config.docs_dir}. Confirm
dataset path before continuing."
            )

        LOGGER.debug("Starting corpus ingestion from %s (%d files)",
self.config.docs_dir, len(pdf_paths))

        parser = (
            MetadataParser.parse_hansard_filename
            if evidence_style == "Hansard"
            else MetadataParser.parse_filename
        )

        records = []
        for pdf_path in pdf_paths:
            submission_id, submitter, submitter_type =
parser(pdf_path.name)
            raw_text, page_count, extraction_ok =
self.extract_pdf_text(pdf_path)

            records.append(
                {
                    "file_name": pdf_path.name,
                    "submission_id": submission_id,
                    "submitter": submitter,
                    "submitter_type": submitter_type,
                    "evidence_style": evidence_style,
                    "n_pages": page_count,
                    "extraction_ok": extraction_ok,
                    "raw_text": raw_text,
                }
            )

        corpus_df = pd.DataFrame(records)
        corpus_df["raw_text"] = corpus_df["raw_text"].fillna("")
        corpus_df["char_count"] = corpus_df["raw_text"].str.len()
        corpus_df["word_count_raw"] =
corpus_df["raw_text"].str.split().str.len()

        filtered_df = corpus_df[corpus_df["word_count_raw"] >=
self.config.min_word_count].reset_index(drop=True)
        LOGGER.debug("Corpus ingestion complete: %d usable documents
(evidence_style=%s)", len(filtered_df), evidence_style)
        return filtered_df
```

0.5 Text preprocessing class and helper functions

Define preprocessing utilities, chart helpers, and reusable analysis functions.

In [7]:
```
class TextPreprocessor:
    """Encapsulates token cleaning, phrase detection, and later
lemmatization rules."""

    def __init__(self, domain_stopwords: set[str], punct_set:
set[str]):
        """Build stopword, punctuation, and lemmatization
resources."""
        base_stopwords = set(stopwords.words("english"))
        self.stop_words = base_stopwords.union(domain_stopwords)
        self.punct_set = punct_set
        self.lemmatizer = WordNetLemmatizer()

    @staticmethod
    def map_pos_tag(tag: str) -> str:
        """Map Penn Treebank tags to the WordNet tags used by the
lemmatizer."""
        if tag.startswith("J"):
            return wn.ADJ
        if tag.startswith("V"):
            return wn.VERB
        if tag.startswith("R"):
            return wn.ADV
        return wn.NOUN

    @staticmethod
    def merge_entities(text: str, nlp_model=None):
        """Merge named entities so multiword mentions survive
downstream tokenization."""
        model = nlp_model if nlp_model is not None else NLP
        if model is None:
            return None

        doc = model(str(text))
        if doc.ents:
            with doc.retokenize() as retokenizer:
                for ent in doc.ents:
                    retokenizer.merge(ent)
        return doc

    @staticmethod
    def normalize_token(token_text: str) -> str:
        """Normalize one token while preserving merged entity
phrases with underscores."""
        normalized = re.sub(r"[^a-z_\s]", "",
token_text.lower()).strip()
        return re.sub(r"\s+", "_", normalized).strip("_")

    def clean_tokens(self, text: str) -> list[str]:
        """Convert raw text into normalized tokens without entity
merging."""
        max_runon_len = 18
        if NLP is not None:
```

```
            doc = NLP(str(text))
            cleaned_tokens = []

            for token in doc:
                normalized = self.normalize_token(token.text)
                if not normalized or len(normalized) < 3:
                    continue
                if len(normalized) > max_runon_len:
                    continue
                if token.pos_ not in {"NOUN", "PROPN", "ADJ",
"VERB"}:
                    continue
                if normalized in self.stop_words or normalized in
self.punct_set:
                    continue

                cleaned_tokens.append(normalized)

            return cleaned_tokens

        tokens = word_tokenize(text.lower())
        tagged_tokens = pos_tag(tokens)
        return [
            token for token, tag in tagged_tokens
            if token.isalpha()
            and len(token) >= 3
            and len(token) <= max_runon_len
            and token not in self.stop_words
            and token not in self.punct_set
            and tag.startswith(("NN", "JJ", "VB"))
        ]

    @staticmethod
    def fit_bigram_model(token_sequences: Iterable[list[str]],
min_count: int = 3, threshold: float = 0.5) -> Phraser:
        """Fit a Gensim Phrases bigram model using NPMI scoring."""
        phrases = Phrases(
            token_sequences,
            min_count=min_count,
            threshold=threshold,
            scoring="npmi",
        )
        return Phraser(phrases)

    @staticmethod
    def fit_pmi_bigram_model(token_sequences: Iterable[list[str]],
min_count: int = 3, threshold: float = 0.5) -> Phraser:
        """Backward-compatible alias for NPMI bigram fitting."""
        return TextPreprocessor.fit_bigram_model(token_sequences,
min_count=min_count, threshold=threshold)

    @staticmethod
    def apply_phrase_model(tokens: list[str], phrase_model: Phraser)
```

```
-> list[str]:
        """Apply a trained Gensim phrase model to one token list."""
        if not tokens:
            return []
        return list(phrase_model[tokens])

    @staticmethod
    def generate_trigram_tokens(tokens: list[str]) -> list[str]:
        """Generate trigrams via nltk.trigrams and return
underscore-joined strings."""
        if len(tokens) < 3:
            return []

        trigram_tuples = list(trigrams(tokens))
        return ["_".join(tri) for tri in trigram_tuples]

    @staticmethod
    def combine_phrase_tokens(bigram_tokens: list[str],
trigram_tokens: list[str]) -> list[str]:
        """Merge bigram and trigram token lists while deduplicating
unigram overlap."""
        merged = []
        seen_unigrams = set()

        for token in list(bigram_tokens or []) + list(trigram_tokens
or []):
            if token.count("_") == 0:
                if token in seen_unigrams:
                    continue
                seen_unigrams.add(token)
            merged.append(token)

        return merged

    def lemmatize_tokens(self, tokens: list[str]) -> list[str]:
        """Lemmatize an already cleaned or phrase detected token
list after collocation analysis."""
        if not tokens:
            return []

        tagged_tokens = pos_tag(tokens)
        return [
            self.lemmatizer.lemmatize(token, self.map_pos_tag(tag))
            for token, tag in tagged_tokens
        ]
```

In [8]:
```
class TextAnalytics:
    """Collect reusable analytics helpers so notebook methods stay
class scoped."""

    @staticmethod
    def compute_ngram_counts(token_sequences: Iterable[list[str]],
n: int = 2) -> Counter:
        """Compute n gram frequencies from tokenized documents."""
        return Counter(gram for tokens in token_sequences for gram
in ngrams(tokens, n))

    @staticmethod
    def extract_top_terms_per_topic(model:
LatentDirichletAllocation, feature_names: np.ndarray, top_n: int =
10) -> pd.DataFrame:
        """Return a dataframe containing top terms for each topic in
an LDA model."""
        topic_rows = []
        for topic_idx, topic_weights in
enumerate(model.components_):
            top_indices = topic_weights.argsort()[::-1][:top_n]
            topic_rows.append(
                {
                    "topic": f"Topic {topic_idx + 1}",
                    "top_terms": ",
".join(feature_names[top_indices]),
                }
            )
        return pd.DataFrame(topic_rows)

    @staticmethod
    def extract_top_terms_per_cluster(matrix, labels: np.ndarray,
feature_names: np.ndarray, top_n: int = 10,
                                      cluster_prefix: str =
"Cluster") -> pd.DataFrame:
        """Return a dataframe containing top weighted terms for each
document cluster."""
        matrix_dense = matrix.toarray() if hasattr(matrix,
"toarray") else np.asarray(matrix)
        labels_array = np.asarray(labels)
        cluster_rows = []

        for cluster_id in sorted(np.unique(labels_array)):
            cluster_matrix = matrix_dense[labels_array ==
cluster_id]
            if cluster_matrix.size == 0:
                continue

            mean_weights = cluster_matrix.mean(axis=0)
            top_indices = np.argsort(mean_weights)[::-1][:top_n]
            cluster_rows.append(
                {
                    "cluster": f"{cluster_prefix} {int(cluster_id) +
```

```
1}",
                    "n_docs": int(cluster_matrix.shape[0]),
                    "top_terms": ",
".join(feature_names[top_indices]),
                }
            )
        return pd.DataFrame(cluster_rows)

    @staticmethod
    def get_coherence_score(
        lemmatized_docs: list[list[str]],
        lda_model: LatentDirichletAllocation,
        feature_names: np.ndarray,
        top_n: int = 10,
    ) -> float:
        """Score an sklearn LDA model with Gensim c_v coherence."""
        words_per_topic = []
        for topic_weights in lda_model.components_:
            top_indices = topic_weights.argsort()[::-1][:top_n]
            words_per_topic.append([feature_names[idx] for idx in
top_indices])

        dictionary = Dictionary(lemmatized_docs)
        if len(dictionary) == 0:
            return np.nan

        coherence_model = CoherenceModel(
            topics=words_per_topic,
            texts=lemmatized_docs,
            dictionary=dictionary,
            coherence="c_v",
        )
        return coherence_model.get_coherence()

    @staticmethod
    def extract_regex_counts(text_series: pd.Series, pattern: str) -
> pd.Series:
        """Extract regex matches from text series and return sorted
frequency counts."""
        matches = text_series.str.findall(pattern)
        flattened = [item for sublist in matches for item in
sublist]
        return pd.Series(flattened).value_counts()
```

In [9]:
```
class Visuals:
    """Reusable plotting helper for consistent chart styling and
labeling."""

    def __init__(self, palette: dict[str, str]):
        """Initialize chart helper with a shared color palette."""
        self.palette = palette

    def bar(self, data: pd.DataFrame, x: str, y: str, title: str,
xlabel: str, ylabel: str, color: str | None = None,
            hue: str | None = None, palette=None, figsize:
tuple[int, int] = (10, 6), ax=None,
            gradient_by_value: bool = True, alpha_min: float = 0.25,
alpha_max: float = 0.95):
        """Draw a configurable vertical bar chart with optional
value based opacity gradient."""
        if ax is None:
            _, ax = plt.subplots(figsize=figsize)
            show_plot = True
        else:
            show_plot = False

        sns.barplot(data=data, x=x, y=y, color=color, hue=hue,
palette=palette, ax=ax)
        ax.set_title(title)
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)

        # Apply a gradient where higher values are rendered with
higher opacity.
        if gradient_by_value and ax.patches:
            bar_values = [max(patch.get_width(), patch.get_height())
for patch in ax.patches]
            min_val = min(bar_values)
            max_val = max(bar_values)

            for patch, value in zip(ax.patches, bar_values):
                if max_val == min_val:
                    alpha = (alpha_min + alpha_max) / 2
                else:
                    scale = (value - min_val) / (max_val - min_val)
                    alpha = alpha_min + (scale * (alpha_max -
alpha_min))
                patch.set_alpha(alpha)

        if show_plot:
            plt.tight_layout()
            plt.show()

    def hist(self, series: pd.Series, title: str, xlabel: str,
ylabel: str, bins: int = 30,
             color: str | None = None, edgecolor: str | None = None,
figsize: tuple[int, int] = (10, 6), ax=None):
```

```
        """Draw a configurable histogram from a series."""
        if ax is None:
            _, ax = plt.subplots(figsize=figsize)
            show_plot = True
        else:
            show_plot = False

        sns.histplot(series, bins=bins, color=color,
edgecolor=edgecolor, ax=ax)
        ax.set_title(title)
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)

        if show_plot:
            plt.tight_layout()
            plt.show()

    def box(self, data: pd.DataFrame, x: str, y: str, title: str,
xlabel: str, ylabel: str,
            palette=None, figsize: tuple[int, int] = (10, 6),
ax=None):
        """Draw a configurable boxplot from dataframe columns."""
        if ax is None:
            _, ax = plt.subplots(figsize=figsize)
            show_plot = True
        else:
            show_plot = False

        sns.boxplot(data=data, x=x, y=y, palette=palette, ax=ax)
        ax.set_title(title)
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)

        if show_plot:
            plt.tight_layout()
            plt.show()

    def scatter(self, data: pd.DataFrame, x: str, y: str, title:
str, xlabel: str, ylabel: str,
                hue: str | None = None, palette=None, alpha: float =
0.7,
                figsize: tuple[int, int] = (10, 6), ax=None):
        """Draw a configurable scatterplot from dataframe
columns."""
        if ax is None:
            _, ax = plt.subplots(figsize=figsize)
            show_plot = True
        else:
            show_plot = False

        sns.scatterplot(data=data, x=x, y=y, hue=hue,
palette=palette, alpha=alpha, ax=ax)
        ax.set_title(title)
```

```
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)

        if show_plot:
            plt.tight_layout()
            plt.show()

    def bigram_network(self, tokens: list[str], title: str, n: int =
50, figsize: tuple[int, int] = (16, 12)):
        """Draw a bigram network graph from existing phrase tokens
using the shared palette."""
        bigram_freq = Counter(
            tuple(token.split("_"))
            for token in tokens
            if isinstance(token, str) and token.count("_") == 1
        )
        if not bigram_freq:
            print("No phrase-based bigrams available to draw.")
            return

        graph = nx.Graph()
        edge_weights = []
        for bigram, count in bigram_freq.most_common(n):
            graph.add_edge(bigram[0], bigram[1], weight=count)
            edge_weights.append(count)

        if not edge_weights:
            print("No bigrams available to draw.")
            return

        edge_weights_array = np.array(edge_weights, dtype=float)
        if edge_weights_array.max() == edge_weights_array.min():
            edge_widths = np.full_like(edge_weights_array, 3.0)
        else:
            edge_widths = 1 + (edge_weights_array -
edge_weights_array.min()) * 5 / (
                edge_weights_array.max() - edge_weights_array.min()
            )

        edge_cmap = LinearSegmentedColormap.from_list(
            "uts_bigram_edges",
            [self.palette["light_grey"], self.palette["blue"]],
        )
        norm = Normalize(vmin=edge_weights_array.min(),
vmax=edge_weights_array.max())
        edge_colors = edge_cmap(norm(edge_weights_array))

        fig, ax = plt.subplots(figsize=figsize)
        pos = nx.spring_layout(graph, k=0.5, iterations=50, seed=42)

        nx.draw_networkx_nodes(
            graph,
            pos,
```

```
            node_size=1000,
            node_color=self.palette["light_grey"],
            edgecolors=self.palette["dark_grey"],
            linewidths=1.0,
            ax=ax,
        )
        nx.draw_networkx_edges(
            graph,
            pos,
            width=edge_widths,
            edge_color=edge_colors,
            alpha=0.9,
            arrows=False,
            ax=ax,
        )
        nx.draw_networkx_labels(
            graph,
            pos,
            font_size=10,
            font_weight="bold",
            font_color=self.palette["dark_grey"],
            ax=ax,
        )

        ax.set_title(title, color=self.palette["dark_grey"])
        ax.axis("off")

        divider = make_axes_locatable(ax)
        cax = divider.append_axes("right", size="5%", pad=0.05)
        sm = plt.cm.ScalarMappable(cmap=edge_cmap, norm=norm)
        sm.set_array([])
        cbar = plt.colorbar(sm, cax=cax)
        cbar.set_label("Edge Weight (Bigram Frequency)",
color=self.palette["dark_grey"])

cbar.ax.yaxis.set_tick_params(color=self.palette["dark_grey"])
        plt.setp(cbar.ax.get_yticklabels(),
color=self.palette["dark_grey"])

        plt.tight_layout()
        plt.show()

    def wordcloud(self, text: str, title: str, width: int = 1200,
height: int = 500, max_words: int = 150,
                  colormap: str = "Blues", background_color: str |
None = None, figsize: tuple[int, int] = (14, 6)):
        """Draw a word cloud from input text with consistent
notebook styling."""
        cloud = WordCloud(
            width=width,
            height=height,
            max_words=max_words,
            colormap=colormap,
```

```
            collocations=False,
            background_color=background_color or
self.palette["white"],
        ).generate(text)

        plt.figure(figsize=figsize)
        plt.imshow(cloud, interpolation="bilinear")
        plt.axis("off")
        plt.title(title, color=self.palette["dark_grey"])
        plt.tight_layout()
        plt.show()
```

1. Introduction

This report analyses public submissions to the Australian parliamentary inquiry into the conduct of the 2025 federal election. Parliamentary inquiries play an important role in democratic policy development by collecting evidence from individuals and organisations and translating that evidence into recommendations for parliament. Using Natural Language Processing (NLP), this project transforms a large PDF corpus of submissions into interpretable insights about dominant concerns, recurring themes, and language patterns in public perspectives.

The analysis aims to answer three core questions:

- **What topics and issues appear most frequently across submissions?**
- **What are the most frequent inquiries raised by the submissions, the Hansard transcripts, and both combined?**
- **Is there any common topic covered by the submissions and the Hansard transcripts?**

To address these questions, the workflow combines text extraction, preprocessing, exploratory analysis, and modeling with visual interpretation.

2. Data Ingestion

Section Focus

This section loads two complementary evidence streams from the Joint Standing Committee on Electoral Matters (JSCEM) inquiry into the 2025 federal election, extracts document text, and builds a unified corpus table for downstream NLP analysis.

Data Sources

Evidence stream	Folder	Files	Source
Written evidence (Submissions)	`docs/submissions/`	299 PDFs	[JSCEM Submissions page](https://www.aph.gov.au/Parliamentary_Business/Committees/Joint/Electoral_Matters/2(
Oral evidence (Hansard transcripts)	`docs/hansard_transcripts/`	7 PDFs	[JSCEM Public Hearings page](https://www.aph.gov.au/Parliamentary_Business/Committees/Joint/Electoral_Matters/2(

Why two sources? Submissions and Hansard transcripts capture different phases of the same parliamentary inquiry. Together they provide a richer and more complete picture of public discourse around electoral reform.

The Relationship: Written vs. Oral Evidence

- **Submissions (Written Evidence):** A broad "net" cast to the public, allowing any individual, organisation, or government agency to send in a formal document detailing their views, data, or experiences.
- **Public Hearings (Oral Evidence / Hansard):** The Committee selects specific witnesses from the pool of people who provided submissions to speak in person. This allows Members of Parliament (MPs) to ask questions, challenge claims, and dig deeper into the most important issues.

High-Level Pipeline: From Inquiry to Law

1. **Announcement & Terms of Reference** -- Parliament defines exactly what will be investigated (e.g., "The conduct of the 2025 Election").
2. **Call for Submissions** -- The public is invited to send in written documents.
3. **Review & Selection** -- The Committee staff (Secretariat) reads all submissions and identifies key themes and important witnesses.
4. **Public Hearings** -- Selected witnesses testify in person; transcripts (Hansard) are published.
5. **Drafting the Report** -- The Committee debates the evidence and writes a final report containing specific Recommendations.
6. **Tabling the Report** -- The report is made public in both the House of Representatives and the Senate.
7. **Government Response** -- The Prime Minister and Cabinet have approximately six months to formally accept or reject the recommendations.
8. **Legislative Action** -- If accepted, the government introduces a Bill to Parliament to officially change the law.

Key Stakeholders and Their Roles

Stakeholder	Category	Role
JSCEM Committee	The Investigators	A group of MPs and Senators who lead the inquiry, question witnesses, and decide on recommendations.
Committee Secretariat	The Engines	Non-partisan parliamentary staff who organise hearings, manage submissions, and draft the final report.
The AEC	The Experts	The Australian Electoral Commission provides technical data on how the election was run and advises if proposed changes are feasible.
Witnesses / Submitters	The Evidence	Citizens, academic experts, and political parties who provide the raw data and personal stories the inquiry is based on.
The Government	The Decision-Makers	The Prime Minister and Cabinet, who decide which recommendations will actually become government policy.
The Senate	The Gatekeepers	The upper house of Parliament that debates and votes on any new laws resulting from the inquiry.

What This Ingestion Step Delivers

- One row per document (submission **or** Hansard transcript).
- An **`evidence_style`** column (`Submission` or `Hansard`) so every later analysis can compare or combine the two evidence streams.
- Metadata parsed from filenames (submission ID or hearing date, submitter label, submitter type).
- Extraction diagnostics (page count, extraction status).
- Text-length features (character and word counts).
- A single `corpus_df` dataframe used in all subsequent analysis sections.

In [10]:
```
# 2.0 Data ingestion execution: build corpus dataframe from
submissions + Hansard

preprocessor = TextPreprocessor(domain_stopwords=DOMAIN_STOPWORDS,
punct_set=PUNCT_SET)
visuals = Visuals(UTS_PALETTE)

sub_service =
CorpusIngestionService(CorpusConfig(docs_dir=SUBMISSIONS_DIR,
min_word_count=1))
hansard_service =
CorpusIngestionService(CorpusConfig(docs_dir=HANSARD_DIR,
min_word_count=1))

submissions_df =
sub_service.read_corpus(evidence_style="Submission")
hansard_df = hansard_service.read_corpus(evidence_style="Hansard")
corpus_df = pd.concat([submissions_df, hansard_df],
ignore_index=True)

print(f"Submission documents:  {len(submissions_df)}")
print(f"Hansard transcripts:   {len(hansard_df)}")
print(f"Total corpus:          {len(corpus_df)}")
print(f"\nEvidence style counts:")
print(corpus_df["evidence_style"].value_counts())
print(f"\nSubmitter type counts:")
print(corpus_df["submitter_type"].value_counts())

corpus_df.head(5)
```

```
MuPDF error: library error: zlib error: incorrect header check

MuPDF error: library error: FT_New_Memory_Face(BCDGEE+ArialMT):
unknown file format

MuPDF error: syntax error: Failed to read JPX header

MuPDF error: library error: zlib error: incorrect header check

MuPDF error: library error: zlib error: incorrect header check

MuPDF error: library error: FT_New_Memory_Face(AAAAAI+ArialMT):
unknown file format

Submission documents:  299
Hansard transcripts:   7
Total corpus:          306

Evidence style counts:
evidence_style
Submission    299
Hansard         7
Name: count, dtype: int64
```

```
Submitter type counts:
submitter_type
People, including fictional                132
Withheld                                    99
Companies, Agencies, Institutions, etc.     53
Unknown                                     15
Hansard                                      7
Name: count, dtype: int64
```

Out[10]:

	file_name	submission_id	submitter	submitter_type	ev
0	Sub001 - Bob McMullan.pdf	Sub001	Bob McMullan	People, including fictional	Su
1	Sub002 - Dr Sheppard.pdf	Sub002	Dr Sheppard	People, including fictional	Su
2	Sub003 - Grattan Institute.pdf	Sub003	Grattan Institute	Companies, Agencies, Institutions, etc.	Su
3	Sub004 - Australian Electoral Commission.pdf	Sub004	Australian Electoral Commission	Companies, Agencies, Institutions, etc.	Su
4	Sub005 - The Australia Institute.pdf	Sub005	The Australia Institute	Companies, Agencies, Institutions, etc.	Su

3. Exploratory Data Analysis

Section Focus

This section establishes high level corpus behavior before advanced modeling.

3.1 Corpus overview visuals

This subsection provides a high level overview of the corpus dataframe before detailed token analysis. The statistics table and charts summarize corpus composition, document scale, and key structural relationships across submissions.

The main concept here is **descriptive statistics**: simple measures such as count, mean, median, minimum, maximum, and spread. These do not explain themes yet, but they tell us whether the corpus is balanced, skewed, or dominated by extreme cases.

In [11]:
```
overview_stats = corpus_df[["n_pages", "char_count",
"word_count_raw"]].describe()
total_values = corpus_df[["n_pages", "char_count",
"word_count_raw"]].sum().rename("sum")
overview_stats_with_sum = overview_stats.copy()
overview_stats_with_sum.loc["sum"] = total_values

# Drop the 'count' column, then transpose and avoid scientific
notation
overview_stats_no_count = overview_stats_with_sum.drop("count",
axis=0)
overview_stats_t = overview_stats_no_count.transpose()
with pd.option_context('display.float_format', '{:,.0f}'.format):
    display(overview_stats_t)
```

	mean	std	min	25%	50%	75%	
n_pages	8	14	1	2	3	7	120
char_count	17,566	39,443	52	2,114	5,352	12,640	258
word_count_raw	2,757	6,437	8	349	844	1,957	43,

The original dataset consisted of **299 submissions** and **7 Hansard transcripts** (306 documents total). However, 23 submissions were removed because their text could not be reliably extracted from the source PDFs, leaving a working corpus of **276 submissions** and **7 Hansard transcripts** (**283 documents** in total).

Document length varies substantially across all three measures (`n_pages` , `char_count` , and `word_count_raw`). This is visible in the large standard deviations and very high maximum values.

A typical document is relatively short (median of **3 pages**, **5,803 characters**, and **919 words**), while the means are noticeably higher (**8 pages**, **18,979 characters**, and **2,979 words**). This gap suggests that a smaller number of very long documents pull the averages upward, with extremes reaching **120 pages**, **258,733 characters**, and **43,400 words**.

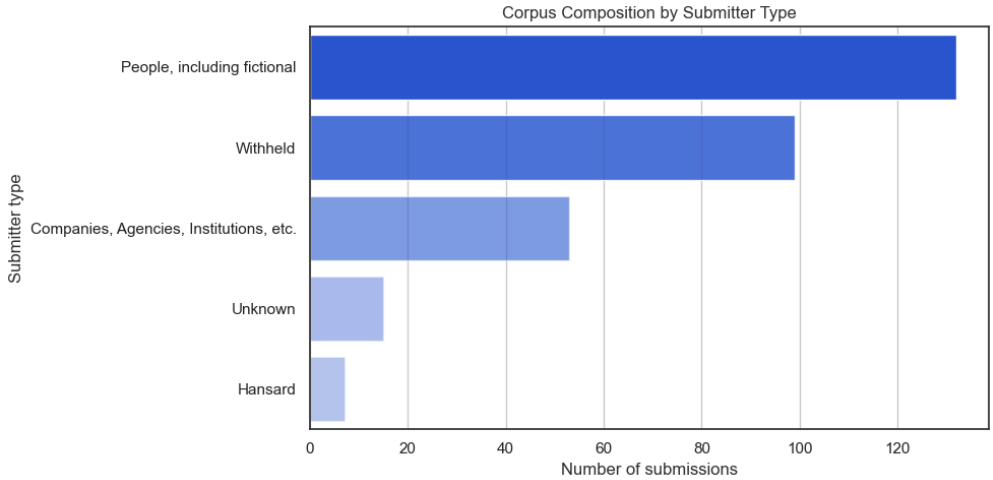
Overall, the distribution is **right-skewed**: most documents are short to medium length, and a limited set of long, information dense outliers extend the upper tail.

3.2 Corpus composition by submitter type

This chart shows how many submissions come from each inferred submitter category.

The key concept here is **metadata based grouping**. Instead of looking only at document text, the notebook also groups submissions by information inferred from filenames and entity labels, which helps frame later comparisons across different kinds of submitters.

In [12]:
```
submitter_counts = (
    corpus_df["submitter_type"]
    .value_counts()
    .rename_axis("submitter_type")
    .reset_index(name="count")
)

visuals.bar(
    data=submitter_counts,
    x="count",
    y="submitter_type",
    title="Corpus Composition by Submitter Type",
    xlabel="Number of submissions",
    ylabel="Submitter type",
    color=UTS_PALETTE["blue"],
    gradient_by_value=True,
    alpha_min=0.35,
    alpha_max=1.0,
    figsize=(10, 5),
)
```



The corpus is clearly not evenly distributed across submitter types, which means this is not a balanced dataset in the statistical sense. Individual submissions and withheld submissions make up a large share of the corpus, while organisation submissions are fewer in number.

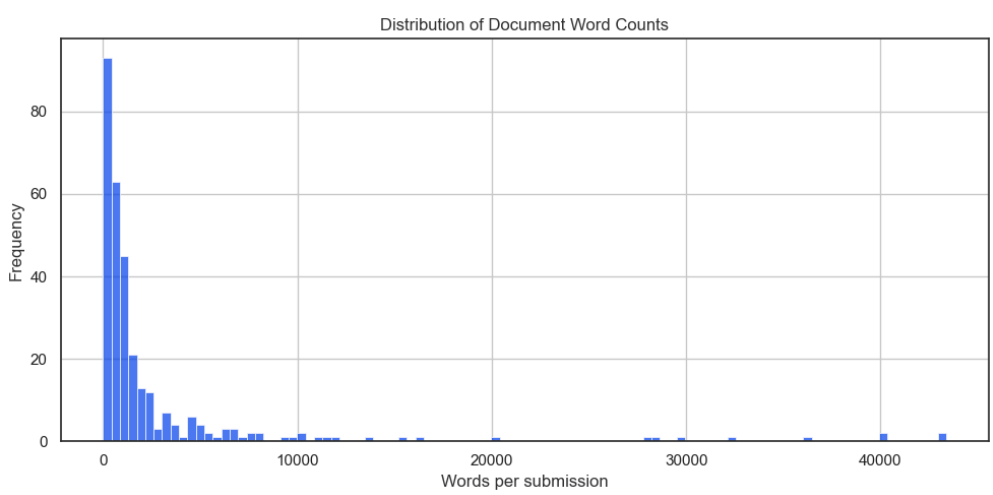
3.3 Distribution of document word counts

This histogram shows the spread of document sizes across submissions.

A **histogram** groups numeric values into bins and shows how often values fall into each range. Here, it helps answer a simple but important question: are most submissions roughly similar in length, or is the corpus dominated by a small number of very short or very long documents?

In [13]:

```
visuals.hist(
    series=corpus_df["word_count_raw"],
    bins=100,
    color=UTS_PALETTE["blue"],
    edgecolor=UTS_PALETTE["white"],
    title="Distribution of Document Word Counts",
    xlabel="Words per submission",
    ylabel="Frequency",
    figsize=(10, 5),
)
```



Most submissions cluster in the shorter to small length range, while a relatively small number extend far to the right of the chart as very large documents. That pattern is called **right skew**: the majority of values are lower, but a minority of very large cases stretch the distribution upward.

In [14]:
```
# Find and show the entire row for the document with the largest
word count
max_word_count_row =
corpus_df.loc[corpus_df["word_count_raw"].idxmax()]
print("Document with the largest word count (entire row):")
display(max_word_count_row.to_frame().T)

# Find and show the entire row for the document with the smallest
word count
min_word_count_row =
corpus_df.loc[corpus_df["word_count_raw"].idxmin()]
print("Document with the smallest word count (entire row):")
display(min_word_count_row.to_frame().T)
```

```
Document with the largest word count (entire row):
```

	file_name	submission_id	submitter	submitte
301	Electoral Matters Joint Committee_2025_11_07.pdf	Hansard_2025-11-07	Electoral Matters Joint Committee	Hansard

```
Document with the smallest word count (entire row):
```

	file_name	submission_id	submitter	submitter_type	eviden
14	Sub015 - Name withheld.pdf	Sub015	Name withheld	Withheld	Submis

Looking directly at the largest and smallest submissions makes the skewness in the histogram concrete. The longest document shows how much detail a single submission can contribute, while the shortest document shows that some cases may function more like brief statements than full reports.

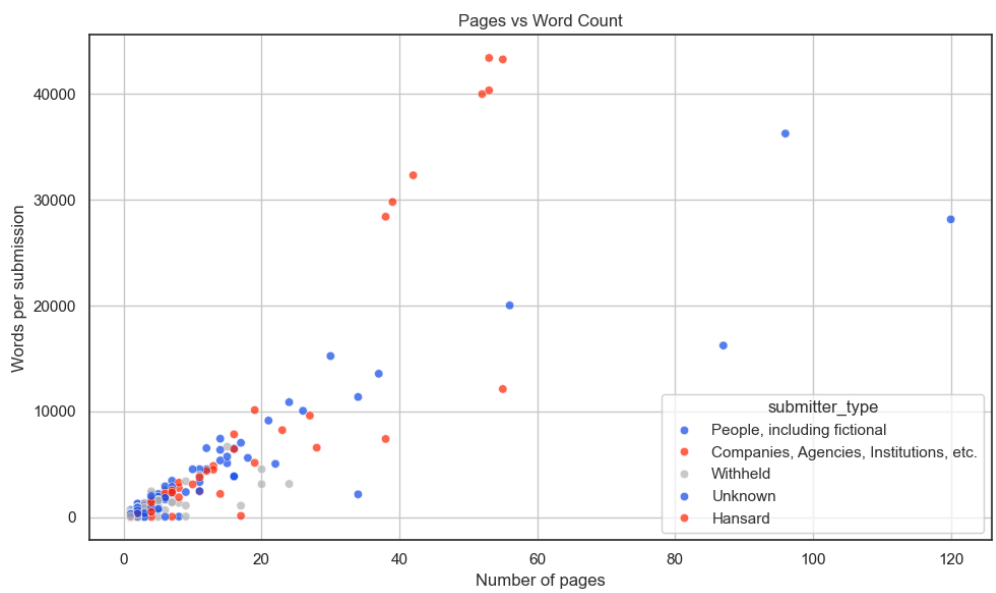
3.4 Relationship between pages and word counts

This scatter plot compares page counts with total words to show document scale relationships.

A **scatter plot** is useful when we want to see whether two numeric variables move together. Here, the question is whether the physical size of a submission in pages aligns with the amount of extracted text in words.

In [15]:
```
visuals.scatter(
    data=corpus_df,
    x="n_pages",
    y="word_count_raw",
    hue="submitter_type",
    palette=[UTS_PALETTE["blue"], UTS_PALETTE["red"],
UTS_PALETTE["mid_grey"]],
    alpha=0.7,
    title="Pages vs Word Count",
    xlabel="Number of pages",
    ylabel="Words per submission",
    figsize=(10, 6),
)
```

```
/var/folders/2f/4qv4v6_s6714t7lb2hc6jtqh0000gn/T/ipykernel_9446
0/1716582564.py:87: UserWarning:
The palette list has fewer values (3) than needed (5) and will
cycle, which may produce an uninterpretable plot.
  sns.scatterplot(data=data, x=x, y=y, hue=hue,
palette=palette, alpha=alpha, ax=ax)
```



The scatter plot reveals a positive but imperfect correlation between page count and word count, confirming that the PDF extraction pipeline has captured text at a scale consistent with the physical size of each document. The relationship is roughly linear for shorter submissions (up to about 20 pages), where each additional page contributes a fairly predictable number of words, but beyond that range the spread increases noticeably, meaning the words per page ratio becomes much less stable in longer documents. Several high page count outliers (above 80 pages) appear in the upper right corner, corresponding to unusually detailed individual submissions that could dominate corpus level statistics if left unweighted, while a handful of moderate page count submissions (around 20 to 35 pages) sit near the y axis baseline, suggesting those PDFs contain substantial non textual content such as images, tables, or scanned pages with limited extractable text. Color coding by `submitter_type` shows that **People (including fictional)** submissions span the widest range on both axes, from single page letters to the longest documents in the dataset, indicating highly variable engagement levels, whereas **Companies, Agencies, and Institutions** tend to cluster in the low to mid range, pointing to more standardized or templated formats.

3.5 Character counts by submitter type

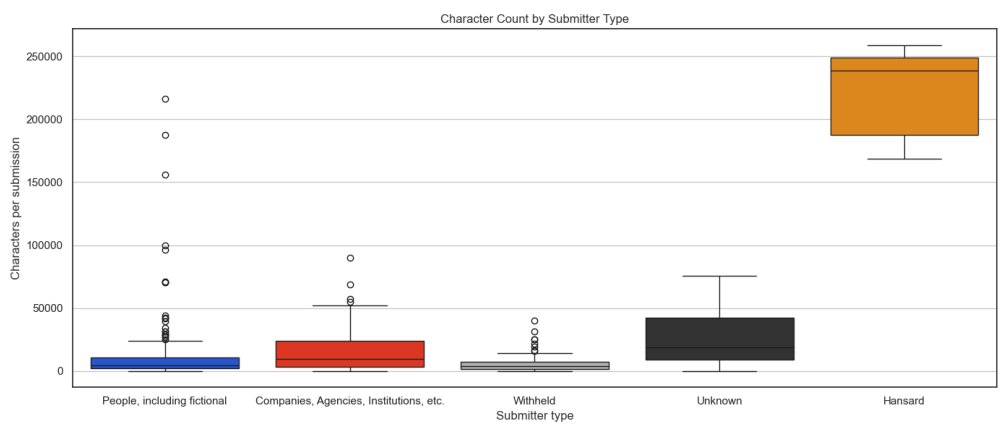
This boxplot compares character count distributions by submitter category.

A **boxplot** summarizes the center, spread, and outliers of a distribution. It is especially useful when document lengths vary widely, because it shows the median and the typical range more clearly than a simple bar chart.

In [16]:

```
# 3.5 Visual: character count by submitter type

visuals.box(
    data=corpus_df,
    x="submitter_type",
    y="char_count",
    palette=[
        UTS_PALETTE["blue"],
        UTS_PALETTE["red"],
        UTS_PALETTE["mid_grey"],
        UTS_PALETTE["dark_grey"],
        "#FF8C00",
    ],
    title="Character Count by Submitter Type",
    xlabel="Submitter type",
    ylabel="Characters per submission",
    figsize=(14, 6),
)
```

```
/var/folders/2f/4qv4v6_s6714t7lb2hc6jtqh0000gn/T/ipykernel_9446
0/1716582564.py:68: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and
will be removed in v0.14.0. Assign the `x` variable to `hue`
and set `legend=False` for the same effect.

  sns.boxplot(data=data, x=x, y=y, palette=palette, ax=ax)
```



Character counts vary substantially both within and across submitter types. Individual submissions (People) have a low median but the widest spread, with extreme outliers exceeding 200,000 characters, while organizational submissions (Companies, Agencies, Institutions) cluster around a higher median with a broader interquartile range, reflecting more consistently substantial report documents. Withheld submissions form the most compact distribution with the lowest median, and the Unknown category, despite few data points, shows a surprisingly high median and wide range. Hansard transcripts stand out as the longest documents by a wide margin: their median exceeds 200,000 characters and even their lower quartile sits above the maximum of every other group, which is expected given that each transcript captures an entire parliamentary hearing session rather than a single written submission. The numerous outliers above the upper whiskers across the non Hansard groups confirm the heavy right skew seen in earlier charts, meaning a small number of very long documents carry a disproportionate share of the total text. The stark length gap between Hansard and all other categories also justifies treating these transcripts separately in downstream analyses (e.g., per document normalization) so that their volume does not dominate corpus wide statistics. This supports the later use of median based summaries, per document normalization, and cleaned token counts rather than relying solely on corpus wide totals.

3.6 Linguistic annotation sample (NER and POS)

This section uses two common NLP annotation tasks to make the language in the corpus more interpretable before preprocessing and modeling.

Named Entity Recognition (NER) identifies spans of text corresponding to real world entities such as people, organisations, places, dates, laws, or quantities. In a corpus like this one, NER is helpful because it shows which institutions, actors, and locations are explicitly named in the submissions.

Part of Speech (POS) tagging assigns a grammatical role to each token, such as noun, verb, adjective, or determiner. It does not tell us the topic directly, but it helps us understand what kind of language dominates the corpus and whether the text is likely to support later thematic analysis.

Named Entity Recognition (NER) sample view

The rendered sample below is included as a qualitative view rather than a full quantitative analysis. Its purpose is to make entity detection visible to the reader so the report does not treat NER as a hidden black box.

In [17]:
```
if NLP is None:
    print("spaCy model unavailable, so inline NER rendering is
skipped in this environment.")
else:
    ner_sample_sentences = []

    for text in corpus_df["raw_text"].fillna("").head(40):
        doc = NLP(str(text)[:6000])
        for sent in doc.sents:
            sent_text = re.sub(r"\s+", " ", sent.text).strip()
            if 50 <= len(sent_text) <= 220 and sent.ents:
                ner_sample_sentences.append(sent_text)
            if len(ner_sample_sentences) >= 10:
                break
        if len(ner_sample_sentences) >= 10:
            break

    if not ner_sample_sentences:
        print("No suitable sentences with named entities were found
in the sampled documents.")
    else:
        ner_sample_docs = [NLP(sentence) for sentence in
ner_sample_sentences]
        ner_sample_entities_df =
MetadataParser.extract_named_entities(
            pd.Series(ner_sample_sentences),
            nlp_model=NLP,
            max_docs=len(ner_sample_sentences),
        )

        print("Sample sentences with detected named entities:")
        spacy.displacy.render(ner_sample_docs, style="ent",
jupyter=True)

        print("Entity labels detected in the rendered sample:")
        display(
            ner_sample_entities_df["label"]
            .value_counts()
            .rename_axis("entity_label")
            .reset_index(name="count")
        )
```

```
Sample sentences with detected named entities:
```

Time to revisit territory representation Introduction The ACT has had the same Senate **ORG** representation with the same anomalous election arrangements for 50 years **DATE** !

During that time the ACT electoral enrolment has increased by 143% **PERCENT** and the size of the State **ORG** representation in the Senate **ORG** has increased by 20% **PERCENT** but the ACT representation has remained unchanged.

The Territories also continue to be disadvantaged by shorter terms and the simultaneous elections provisions which were included in the 1974 **DATE** legislation to establish territory representation.

Submission In 1975 **DATE** the enrolment for the first **ORDINAL** ACT Senate **ORG** election was 115916 **DATE** .

The 1984 **DATE** changes increased the discrepancy between state and territory representation.

State Senate **ORG** representation increased by 20% **PERCENT** but Territory representation remained the same.

Given that the constitutional design of Senate **ORG** representation is based on enhanced representation of smaller jurisdictions it is appropriate to compare trends in the ACT with those of the least populated states.

The population growth in the ACT is significantly greater than the increase in the two **CARDINAL** smallest states, Tasmania **GPE** and South Australia **GPE** .

In the ACT the increase in electoral enrolment has been 143% **PERCENT** since 1975 **DATE** compared to 61% **PERCENT** in Tasmania **GPE** and 65% **PERCENT** in South Australia **GPE** .

Enrolment in the ACT has grown from 45% **PERCENT** of that in Tasmania **GPE** in 1975 **DATE** to 78% **PERCENT** in 2025 **DATE** .

`Entity labels detected in the rendered sample:`

	entity_label	count
0	DATE	8

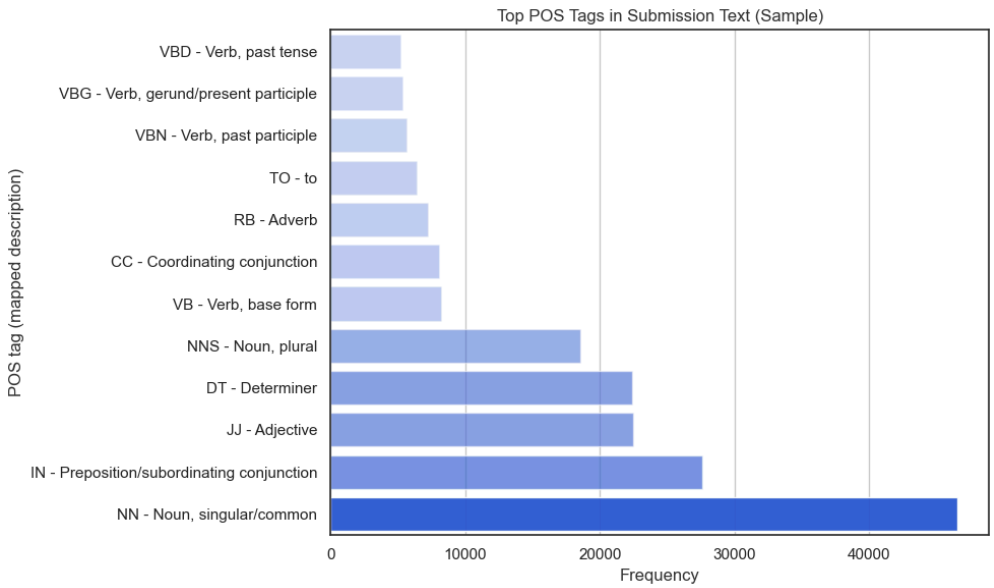
	entity_label	count
1	PERCENT	8
2	ORG	6
3	GPE	5
4	ORDINAL	1
5	CARDINAL	1

The rendered sample confirms that spaCy reliably detects the entity types central to electoral policy text: **ORG** for institutions like the Senate, **GPE** for jurisdictions such as Tasmania and South Australia, **DATE** for legislative time references, and **PERCENT** / **CARDINAL** for the quantitative evidence submitters use to support their arguments. The high density of annotations in just a few sentences shows that the corpus is rich in formal, fact laden language where named entities carry much of the argumentative weight. NER is not perfect, as some labels may be wrong or entities missed, but this qualitative view makes the detection step transparent so that later results built on entity features can be interpreted with appropriate confidence.

Part of Speech (POS) analysis

The chart below tags words in a runtime capped sample of submission text and summarizes the most frequent grammatical categories.

In [18]:
```
pos_max_docs = 120
pos_tags = []

for text in corpus_df["raw_text"].head(pos_max_docs):
    tokens = [token.lower() for token in word_tokenize(str(text)) if
token.isalpha()]
    if not tokens:
        continue
    pos_tags.extend(tag for _, tag in pos_tag(tokens))

pos_tag_counts = pd.Series(pos_tags,
name="pos_tag").value_counts().reset_index()
pos_tag_counts.columns = ["pos_tag", "count"]
pos_tag_counts["pos_description"] =
pos_tag_counts["pos_tag"].map(PENN_POS_TAG_MAP).fillna("Other/Unknow
n")
pos_tag_counts["pos_label"] = pos_tag_counts["pos_tag"] + " - " +
pos_tag_counts["pos_description"]

top_pos_tags = pos_tag_counts.head(12).sort_values("count",
ascending=True)

LOGGER.debug(
    "POS tagging complete: docs=%d, tagged_tokens=%d",
    min(pos_max_docs, len(corpus_df)),
    len(pos_tags),
)

visuals.bar(
    data=top_pos_tags,
    x="count",
    y="pos_label",
    color=UTS_PALETTE["blue"],
    title="Top POS Tags in Submission Text (Sample)",
    xlabel="Frequency",
    ylabel="POS tag (mapped description)",
    figsize=(10, 6),
)

top_pos_tags[["pos_tag", "pos_description", "count"]]
```



	pos_tag	pos_description	count
11	VBD	Verb, past tense	5193
10	VBG	Verb, gerund/present participle	5305
9	VBN	Verb, past participle	5626
8	TO	to	6401
7	RB	Adverb	7208
6	CC	Coordinating conjunction	8055
5	VB	Verb, base form	8228
4	NNS	Noun, plural	18568
3	DT	Determiner	22413
2	JJ	Adjective	22473
1	IN	Preposition/subordinating conjunction	27606
0	NN	Noun, singular/common	46556

Nouns, prepositions, determiners, and adjectives dominate the POS profile. That is typical of formal policy style submissions because these documents spend a lot of time naming institutions, actors, procedures, and issues rather than using conversational or narrative language.

4. Preprocessing

4.1 Prepare cleaned tokens and lexical metrics

This cell converts the raw text into analysis ready form through a sequence of steps, each designed to progressively reduce the dimensionality of the data so that later models work with a smaller, more meaningful vocabulary instead of a noisy, inflated one.

Step 1 — URL cleanup and clean tokens. Before tokenization, URLs are removed from `raw_text` using a regex pattern (matching both `http(s)://` and `www.` links) so web addresses do not pollute lexical features. The `clean_tokens` method then applies POS filtering to keep only nouns, proper nouns, adjectives, and verbs, which are the parts of speech that carry the most topical meaning. Stopwords, punctuation, and non alphabetic tokens are discarded. The result is a focused token list that removes high frequency, low information words and dramatically reduces the number of unique features the corpus contains.

Step 2 — Bigram phrase detection. Gensim's `Phrases` model is used with `scoring="npmi"` to detect collocations in the cleaned token stream. Bigrams that satisfy `min_count=3` and `threshold=0.5` are merged into underscore phrases (for example, `electoral + commission -> electoral_commission`) so the representation keeps multi word concepts as single units while filtering weaker pairs.

Step 3 — Trigram extraction. Trigrams are generated directly from cleaned tokens using `nltk.trigrams` , producing contiguous three word windows that are stored as underscore joined phrase tokens.

Step 4 — Lemmatization. Each token is mapped to its base dictionary form using NLTK's `WordNetLemmatizer` . Penn Treebank POS tags are converted to WordNet tags so the lemmatizer can apply the correct morphological rules (for example, `votes` , `voted` , and `voting` all become `vote` , while adjectives and adverbs are handled by their own rules). By merging inflected variants into a common lemma, this step

compresses the vocabulary even further, ensuring that surface level variation does not artificially inflate feature counts in downstream models like TF-IDF, topic modeling, and clustering.

Finally, the cell computes per document metrics including **cleaned word count** and **lexical diversity** (the ratio of unique tokens to total tokens) so these values are available for all subsequent comparisons.

In [19]:

```
# Remove URLs before tokenization so links do not pollute lexical
features.
corpus_df["text_no_urls"] = (
    corpus_df["raw_text"]
    .fillna("")
    .astype(str)
    .str.replace(r"(?:https?://|www\.)\S+", " ", regex=True)
)

corpus_df["raw_tokens"] = corpus_df["text_no_urls"].apply(
    lambda text: [token.lower() for token in
word_tokenize(str(text)) if token.isalpha()]
)
corpus_df["clean_tokens"] =
corpus_df["text_no_urls"].apply(preprocessor.clean_tokens)

# Remove records where the clean_tokens list is empty
corpus_df = corpus_df[corpus_df["clean_tokens"].str.len() >
0].copy().reset_index(drop=True)

# Build NPMI bigrams with Gensim Phrases and trigrams with
nltk.trigrams.
bigram_model =
preprocessor.fit_bigram_model(corpus_df["clean_tokens"],
min_count=3, threshold=0.5)

# Keep existing column names for downstream compatibility.
corpus_df["pmi_bigram_tokens"] = corpus_df["clean_tokens"].apply(
    lambda tokens: preprocessor.apply_phrase_model(tokens,
bigram_model)
)
corpus_df["pmi_trigram_tokens"] =
corpus_df["clean_tokens"].apply(preprocessor.generate_trigram_tokens
)

# Join bigram and trigram outputs, but avoid duplicated unigrams.
corpus_df["combined_phrase_tokens"] = corpus_df.apply(
    lambda row:
preprocessor.combine_phrase_tokens(row["pmi_bigram_tokens"],
row["pmi_trigram_tokens"]),
    axis=1,
)

corpus_df["lemmatized_tokens"] =
corpus_df["combined_phrase_tokens"].apply(preprocessor.lemmatize_tok
ens)
corpus_df["preprocessed_text"] =
corpus_df["lemmatized_tokens"].apply(" ".join)

corpus_df["clean_word_count"] = corpus_df["clean_tokens"].str.len()
corpus_df["lexical_diversity"] = corpus_df.apply(
    lambda r: (len(set(r["clean_tokens"])) / r["clean_word_count"])
if r["clean_word_count"] > 0 else np.nan,
```

```
    axis=1,
)

raw_all_tokens = [token for tokens in corpus_df["raw_tokens"] for
token in tokens]
all_tokens = [token for tokens in corpus_df["clean_tokens"] for
token in tokens]
pmi_bigram_vocab = {token for tokens in
corpus_df["pmi_bigram_tokens"] for token in tokens}
pmi_trigram_vocab = {token for tokens in
corpus_df["pmi_trigram_tokens"] for token in tokens}
combined_phrase_vocab = {token for tokens in
corpus_df["combined_phrase_tokens"] for token in tokens}
lemmatized_vocab = {token for tokens in
corpus_df["lemmatized_tokens"] for token in tokens}
vocab_size = len(set(all_tokens))
LOGGER.debug(
    "Tokenization complete: raw_vocab_size=%d, clean_vocab_size=%d,
bigram_vocab_size=%d, trigram_vocab_size=%d, combined_vocab_size=%d,
lemmatized_vocab_size=%d, docs=%d",
    len(set(raw_all_tokens)),
    vocab_size,
    len(pmi_bigram_vocab),
    len(pmi_trigram_vocab),
    len(combined_phrase_vocab),
    len(lemmatized_vocab),
    len(corpus_df),
)

corpus_df[["file_name", "clean_tokens", "pmi_bigram_tokens",
"pmi_trigram_tokens", "combined_phrase_tokens",
"lemmatized_tokens"]].head(10)
```

Out[19]:

	file_name	clean_tokens	pmi_bigram_tokens	pm
0	Sub001 - Bob McMullan.pdf	[time, revisit, territory, representation, int...	[time, revisit, territory, representation, int...	[time_revisit_ revisit_territo
1	Sub002 - Dr Sheppard.pdf	[national, university, possible, solutions, mo...	[national, university, possible, solutions, mo...	[national_uni university_po
2	Sub003 - Grattan Institute.pdf	[grattan, institute, september, aruna, kate, g...	[grattan_institute, september, aruna, kate, gr...	[grattan_insti institute_sept

	file_name	clean_tokens	pmi_bigram_tokens	pm
3	Sub004 - Australian Electoral Commission.pdf	[aec, joint, standing, matters, aec, joint, st...	[aec, joint_standing, matters, aec, joint_stan...	[aec_joint_st joint_standin
4	Sub005 - The Australia Institute.pdf	[democracy, thriving, inventive, democracy, me...	[democracy, thriving, inventive, democracy, me...	[democracy_ thriving_inve
5	Sub006 - Andrew Murray.pdf	[secretary, joint, standing, matters, box, hou...	[secretary_joint, standing_matters, box, house...	[secretary_jo joint_standin
6	Sub007 - Genevieve Grant.pdf	[page, written, joint, standing, matters, refe...	[page, written, joint_standing, matters, refer...	[page_writter written_joint_
7	Sub008 - Richard Ure.pdf	[years, epping, first, suburb, sydney, declare...	[years, epping, first, suburb, sydney, declare...	[years_eppin epping_first_
8	Sub009 - Barry Searle.pdf	[barry, searle, sept, time, banned, poll, boot...	[barry, searle, sept, time, banned, poll, boot...	[barry_searle searle_sept_
9	Sub010 - Name withheld.pdf	[regarding, brief, voted, day, polling, ended,...	[regarding, brief, voted, day, polling, ended,...	[regarding_b brief_voted_

This preprocessing step creates several parallel versions of the text, from raw tokens to cleaned tokens, phrase-aware tokens, and finally lemmatized tokens. That layered approach is useful because different tasks need different text representations: simple counts can use `clean_tokens`, phrase analysis needs the bigram output, and modeling benefits from the more normalized `preprocessed_text` field.

In [40]:
```
corpus_df
```

Out[40]:

	file_name	submission_id	submitter	submi
0	Sub001 - Bob McMullan.pdf	Sub001	Bob McMullan	People includi fiction
1	Sub002 - Dr Sheppard.pdf	Sub002	Dr Sheppard	People includi fiction
2	Sub003 - Grattan Institute.pdf	Sub003	Grattan Institute	Compa Agenci Institut etc.
3	Sub004 - Australian Electoral Commission.pdf	Sub004	Australian Electoral Commission	Compa Agenci Institut etc.
4	Sub005 - The Australia Institute.pdf	Sub005	The Australia Institute	Compa Agenci Institut etc.
...	...	...	...	...
278	Electoral Matters Joint Committee_2025_11_07.pdf	Hansard_2025-11-07	Electoral Matters Joint Committee	Hansa
279	Electoral Matters Joint Committee_2025_11_12.pdf	Hansard_2025-11-12	Electoral Matters Joint Committee	Hansa
280	Electoral Matters Joint Committee_2025_11_19.pdf	Hansard_2025-11-19	Electoral Matters Joint Committee	Hansa
281	Electoral Matters Joint Committee_2025_11_20.pdf	Hansard_2025-11-20	Electoral Matters Joint Committee	Hansa

	file_name	submission_id	submitter	submi
282	Electoral Matters Joint Committee_2025_12_10.pdf	Hansard_2025-12-10	Electoral Matters Joint Committee	Hansa

283 rows × 25 columns

4.2 Most common words

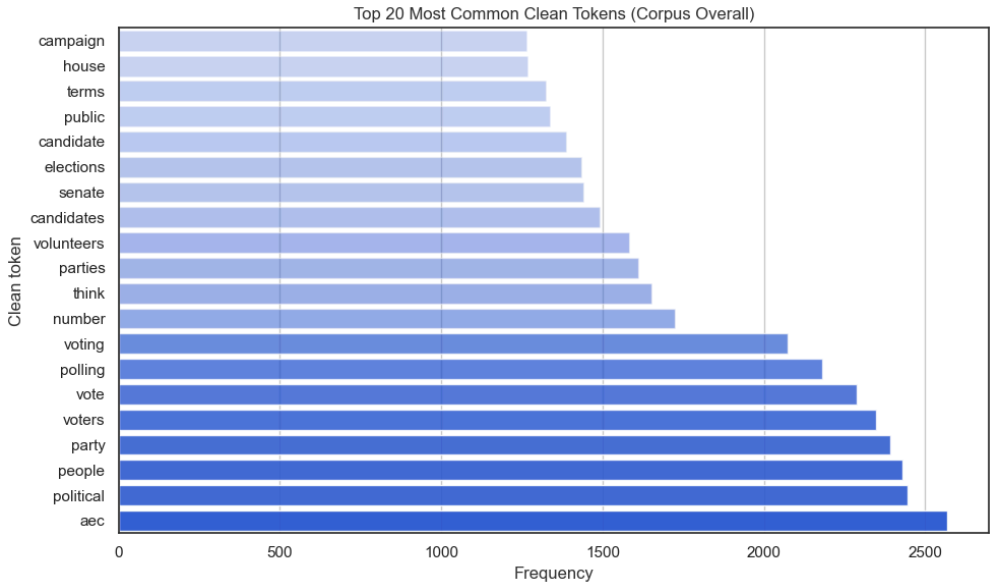
This subsection focuses on visualization only. Using the cleaned tokens prepared in the previous step, it shows the most common terms across the full corpus and within each `submitter_type`.

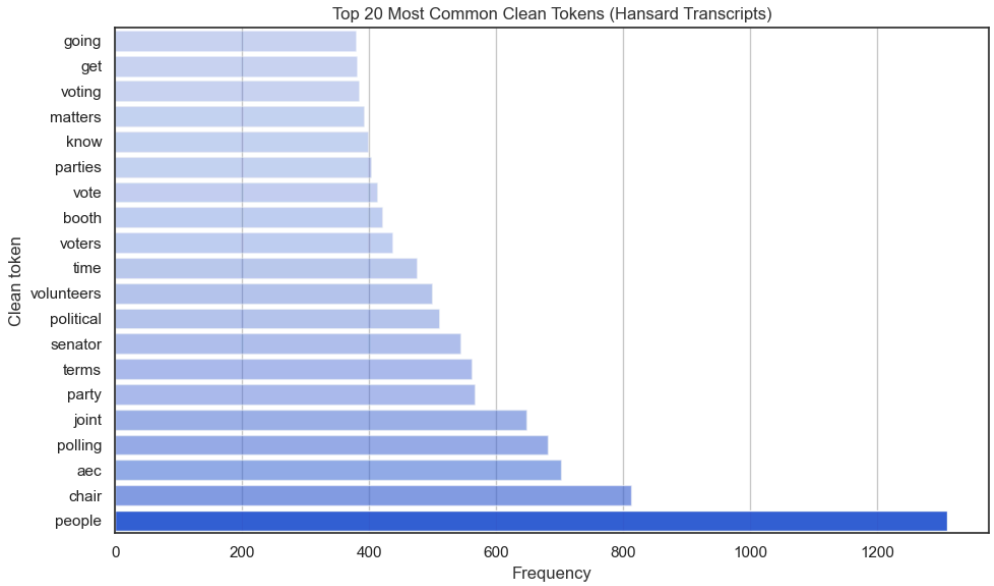
The concept here is **term frequency**: counting how often words appear. Frequency is simple, but it is often the clearest first signal of what the corpus is about once obvious stopwords and formatting noise have been removed.

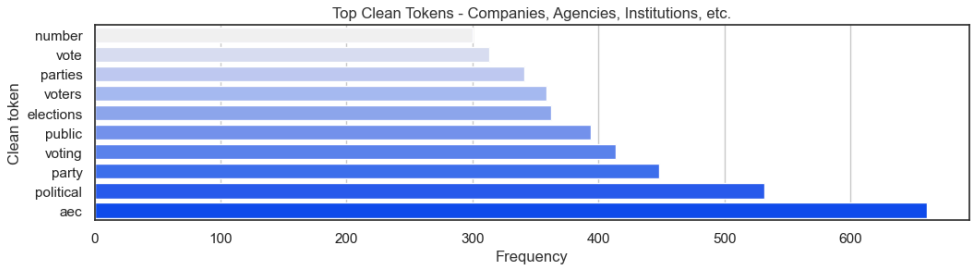
In [20]:

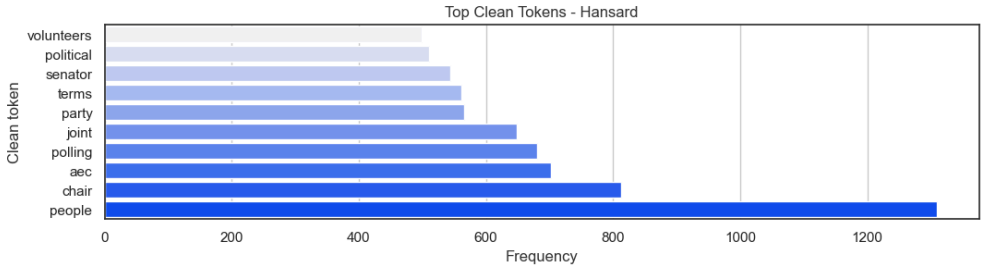
```
# 4.2 Visuals: most common words (clean tokens)

overall_word_counter = Counter(all_tokens)
top_words_df = pd.DataFrame(overall_word_counter.most_common(20),
columns=["word", "count"])
plot_top_words = top_words_df.sort_values("count", ascending=True)

LOGGER.debug("Clean token visualization prepared:
unique_clean_tokens=%d", len(overall_word_counter))

# Chart 1: overall top clean tokens
visuals.bar(
    data=plot_top_words,
    x="count",
    y="word",
    color=UTS_PALETTE["blue"],
    title="Top 20 Most Common Clean Tokens (Corpus Overall)",
    xlabel="Frequency",
    ylabel="Clean token",
    figsize=(10, 6),
)

# Chart 1b: top clean tokens from Hansard transcripts only
HANSARD_EXTRA_STOPS = {"think"}
hansard_tokens = [
    token
    for tokens in corpus_df.loc[corpus_df["evidence_style"] ==
"Hansard", "clean_tokens"]
    for token in tokens
    if token not in HANSARD_EXTRA_STOPS
]
hansard_word_counter = Counter(hansard_tokens)
hansard_top_df = (
    pd.DataFrame(hansard_word_counter.most_common(20), columns=
["word", "count"])
    .sort_values("count", ascending=True)
)

visuals.bar(
    data=hansard_top_df,
    x="count",
    y="word",
    color=UTS_PALETTE["blue"],
    title="Top 20 Most Common Clean Tokens (Hansard Transcripts)",
    xlabel="Frequency",
    ylabel="Clean token",
    figsize=(10, 6),
)

# Top clean tokens by submitter type
rows = []
for submitter_type, group_df in corpus_df.groupby("submitter_type"):
    extra_stops = HANSARD_EXTRA_STOPS if submitter_type == "Hansard"
```

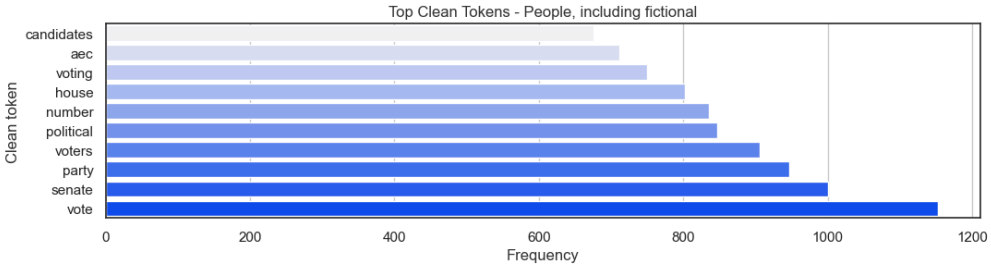
```
else set()
    type_tokens = [
        token
        for tokens in group_df["clean_tokens"]
        for token in tokens
        if token not in extra_stops
    ]
    type_counter = Counter(type_tokens)
    for word, count in type_counter.most_common(10):
        rows.append({"submitter_type": submitter_type, "word": word,
"count": count})

top_words_by_type_df = pd.DataFrame(rows)

# Chart 2: top clean tokens by submitter type (faceted)
submitter_types =
list(top_words_by_type_df["submitter_type"].dropna().unique())
fig, axes = plt.subplots(
    nrows=len(submitter_types),
    ncols=1,
    figsize=(11, max(3 * len(submitter_types), 6)),
    sharex=False,
)

if len(submitter_types) == 1:
    axes = [axes]

for ax, submitter_type in zip(axes, submitter_types):
    type_data = (
        top_words_by_type_df[top_words_by_type_df["submitter_type"]
== submitter_type]
        .sort_values("count", ascending=True)
    )

    sns.barplot(data=type_data, x="count", y="word",
color=UTS_PALETTE["blue"], ax=ax)

    gradient_colors = sns.light_palette(UTS_PALETTE["blue"],
n_colors=len(type_data))
    for bar, color in zip(ax.patches, gradient_colors):
        bar.set_facecolor(color)

    ax.set_title(f"Top Clean Tokens - {submitter_type}")
    ax.set_xlabel("Frequency")
    ax.set_ylabel("Clean token")

plt.tight_layout()
plt.show()
```

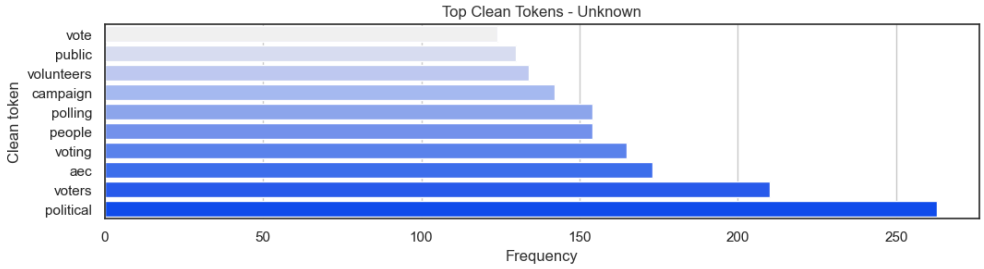

Top 20 Most Common Clean Tokens (Corpus Overall)
campaign
house
terms
public
candidate
elections
senate
candidates
volunteers
parties
think
number
voting
polling
vote
voters
party
people
political
aec
Clean token
0
500
1000
1500
2000
2500
Frequency

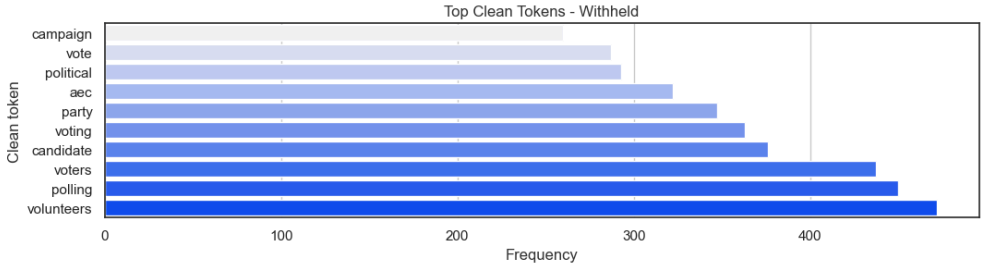

Top 20 Most Common Clean Tokens (Hansard Transcripts)
going
get
voting
matters
know
parties
vote
booth
voters
time
volunteers
political
senator
terms
party
joint
polling
aec
chair
people
Clean token
0
200
400
600
800
1000
1200
Frequency


Top Clean Tokens - Companies, Agencies, Institutions, etc.
Clean token
number
vote
parties
voters
elections
public
voting
party
political
aec
0
100
200
300
400
500
600
Frequency


Top Clean Tokens - Hansard
Clean token
volunteers
political
senator
terms
party
joint
polling
aec
chair
people
0
200
400
600
800
1000
1200
Frequency


Top Clean Tokens - People, including fictional
Clean token
candidates
aec
voting
house
number
political
voters
party
senate
vote
0
200
400
600
800
1000
1200
Frequency


Top Clean Tokens - Unknown
Clean token
vote
public
volunteers
campaign
polling
people
voting
aec
voters
political
0
50
100
150
200
250
Frequency


Top Clean Tokens - Withheld
Clean token
campaign
vote
political
aec
party
voting
candidate
voters
polling
volunteers
0
100
200
300
400
Frequency

Because these charts use `clean_tokens`, the dominant terms are easier to interpret than raw token counts and better reflect substantive inquiry themes rather than formatting noise or stopwords.

Overall corpus. The corpus chart is dominated by **aec** (~2,600 occurrences), confirming the Australian Electoral Commission as the institutional anchor of the inquiry. A cluster of democratic process terms *political*, *people*, *party*, *voters*, *vote*, *polling*, *voting* — forms the next tier (roughly 2,000–2,500 each), reflecting the core subject matter: elections, political participation, and polling logistics. Lower ranked but still prominent terms like *volunteers*, *candidates*, *senate*, *elections*, *public*, and *campaign* fill out the operational vocabulary of electoral administration.

Hansard transcripts. The high-frequency filler *think* (witnesses saying "I think…") was excluded as a Hansard-specific stopword because it reflects oral register rather than substantive content. With that removed, **people** stands out as the clear leader (~1,300), nearly double the next token, confirming that oral testimony centres on public interest and everyday citizens. **chair** (~800) follows, reflecting the procedural turn-taking of committee hearings ("Chair, I would like to…"). A mid-tier cluster — *aec* (~700), *polling* (~680), *joint* (~650), *party* and *terms* (~580), *senator* (~550) — captures the institutional and procedural vocabulary unique to parliamentary discourse. Lower-ranked terms like *volunteers*, *time*, *voters*, *booth*, *vote*, *know*, and *going* introduce an operational and conversational layer that is absent from written submissions, reinforcing that Hansard transcripts blend electoral substance with the informal, deliberative register of live testimony.

Faceted charts by submitter type. The per-group breakdown reveals how each stakeholder frames the inquiry differently:

- **Companies, Agencies, Institutions, etc.** — *aec* leads by a wide margin (~680), followed by *political* and *party*. This group speaks in institutional and regulatory language, foregrounding agencies, policy, and elections administration.
- **Hansard** — with *think* excluded, *people* and *chair* lead, followed by *aec*, *polling*, and *joint*, reflecting the oral, deliberative nature of transcribed hearings and their procedural vocabulary.
- **People, including fictional** — *vote* is the clear leader (~1,150), followed by *senate*, *party*, and *voters*. Individual submitters centre their language on the act of voting itself and the senate electoral system, suggesting grassroots concerns about ballot access and representation.
- **Unknown** — *political* leads (~265), with *voters* and *aec* also prominent. The vocabulary resembles the institutional group but at lower scale, likely representing submissions where the submitter category could not be determined but the content still tracks mainstream inquiry themes.
- **Withheld** — *volunteers* is the top token (~475), a unique emphasis not seen in any other group. *polling*, *voters*, and *candidate* follow. This pattern suggests that withheld submitters are disproportionately concerned with the volunteer workforce at polling

places and candidate level issues, possibly reflecting electoral experience from individuals who preferred anonymity.

4.3 Corpus snapshot statistics

This cell reports compact EDA summary statistics used to frame the visual analysis.

Think of this as a checkpoint in the story: after cleaning the corpus, the notebook pauses to summarize how large, varied, and information rich the text now looks before moving into n grams and models.

In [21]:

```
# 4.3 Snapshot statistics

print("=== EDA Snapshot ===")
print(f"Documents analysed: {len(corpus_df)}")
print(f"Corpus vocabulary size (clean tokens): {vocab_size:,}")
print(f"Median document word count (raw):
{corpus_df['word_count_raw'].median():.0f}")
print(f"Median lexical diversity:
{corpus_df['lexical_diversity'].median():.3f}")
```

```
=== EDA Snapshot ===
Documents analysed: 283
Corpus vocabulary size (clean tokens): 17,304
Median document word count (raw): 919
Median lexical diversity: 0.644
```

4.4 Frequency table preparation (unigrams and NPMI bigrams)

This cell computes unigram frequencies from the cleaned token stream and phrase frequencies from the NPMI bigram output so the following charts can compare raw n grams against detected phrase units.

What these concepts mean:
A **unigram** is a single token.
A **bigram** is a two word sequence.
NPMI (Normalized Pointwise Mutual Information) is a collocation measure that helps identify word combinations that occur together more meaningfully than chance would suggest.

This matters because raw adjacency alone can produce many common but uninformative phrases, whereas NPMI helps highlight combinations that behave more like stable concepts.

In [23]:

```
# 4.4 Frequency tables

raw_unigram_counts = Counter(raw_all_tokens)
clean_unigram_counts = Counter(all_tokens)
raw_bigram_counts =
TextAnalytics.compute_ngram_counts(corpus_df["raw_tokens"], n=2)
raw_trigram_counts =
TextAnalytics.compute_ngram_counts(corpus_df["raw_tokens"], n=3)
phrase_bigram_counts = Counter(
    token.replace("_", " ")
    for tokens in corpus_df["pmi_bigram_tokens"]
    for token in tokens
    if token.count("_") == 1
)
phrase_trigram_counts = Counter(
    token.replace("_", " ")
    for tokens in corpus_df["pmi_trigram_tokens"]
    for token in tokens
    if token.count("_") == 2
)

top_raw_unigrams = pd.DataFrame(raw_unigram_counts.most_common(20),
columns=["term", "count"])
top_clean_unigrams =
pd.DataFrame(clean_unigram_counts.most_common(20), columns=["term",
"count"])

top_raw_bigrams = pd.DataFrame(
    [(" ".join(bg), count) for bg, count in
raw_bigram_counts.most_common(15)],
    columns=["bigram", "count"],
)
top_phrase_bigrams = pd.DataFrame(
    list(phrase_bigram_counts.most_common(15)),
    columns=["bigram", "count"],
)

top_raw_trigrams = pd.DataFrame(
    [(" ".join(tg), count) for tg, count in
raw_trigram_counts.most_common(15)],
    columns=["trigram", "count"],
)
top_phrase_trigrams = pd.DataFrame(
    list(phrase_trigram_counts.most_common(15)),
    columns=["trigram", "count"],
)

print("Top 10 raw unigrams:")
display(top_raw_unigrams.head(10))

print("Top 10 clean unigrams:")
display(top_clean_unigrams.head(10))
```

```
print("Top 10 raw bigrams:")
display(top_raw_bigrams.head(10))

print("Top 10 phrase bigrams:")
display(top_phrase_bigrams.head(10))

print("Top 10 raw trigrams:")
display(top_raw_trigrams.head(10))

print("Top 10 phrase trigrams:")
display(top_phrase_trigrams.head(10))
```

```
Top 10 raw unigrams:
```

	term	count
0	the	48294
1	to	24489
2	of	24459
3	and	21129
4	a	14984
5	in	14823
6	that	13787
7	i	9125
8	for	8491
9	is	8052

```
Top 10 clean unigrams:
```

	term	count
0	aec	2569
1	political	2445
2	people	2430
3	party	2393
4	voters	2349
5	vote	2289
6	polling	2181
7	voting	2075

	term	count
8	number	1725
9	think	1652

```
Top 10 raw bigrams:
```

	bigram	count
0	of the	5897
1	in the	3561
2	to the	2804
3	federal election	2802
4	the federal	2648
5	into the	2387
6	inquiry into	2231
7	election submission	1963
8	the aec	1914
9	for the	1809

```
Top 10 phrase bigrams:
```

	bigram	count
0	house representatives	588
1	pre poll	477
2	polling booth	435
3	political parties	430
4	polling booths	419
5	joint standing	318
6	matters joint	306
7	joint page	304
8	third party	301
9	polling places	299

```
Top 10 raw trigrams:
```

	trigram	count
0	the federal election	2483
1	inquiry into the	2191
2	into the federal	2089
3	federal election submission	1957
4	the number of	772
5	house of representatives	581
6	the house of	472
7	joint standing committee	370
8	committee on electoral	363
9	standing committee on	354

```
Top 10 phrase trigrams:
```

	trigram	count
0	joint standing matters	350
1	joint page matters	304
2	page matters joint	304
3	truth political advertising	199
4	november joint page	166
5	new south wales	165
6	october joint page	89
7	ppvc total votes	82
8	wednesday november joint	82
9	state division ppvc	81

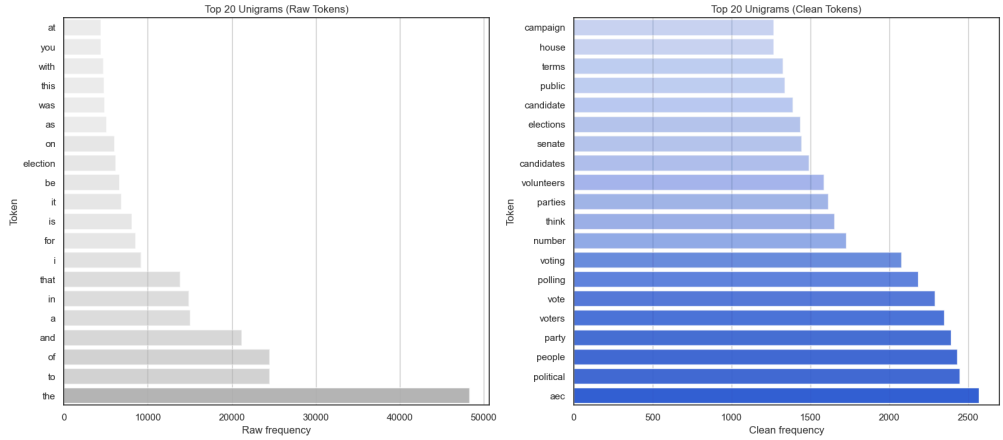
4.5 Top unigram frequency chart

These charts visualize the most common single word terms in separate raw token and cleaned token views so the effect of preprocessing is easy to compare.

A **unigram frequency chart** is one of the simplest NLP views, but it is very useful for storytelling because it shows how the vocabulary changes once cleaning decisions have been applied.

In [24]:

```
# 4.5 Visual: top unigrams

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(16, 7),
constrained_layout=True)

visuals.bar(
    data=top_raw_unigrams.sort_values("count", ascending=True),
    x="count",
    y="term",
    color=UTS_PALETTE["mid_grey"],
    title="Top 20 Unigrams (Raw Tokens)",
    xlabel="Raw frequency",
    ylabel="Token",
    ax=axes[0],
)

visuals.bar(
    data=top_clean_unigrams.sort_values("count", ascending=True),
    x="count",
    y="term",
    color=UTS_PALETTE["blue"],
    title="Top 20 Unigrams (Clean Tokens)",
    xlabel="Clean frequency",
    ylabel="Token",
    ax=axes[1],
)

plt.show()
```



Showing raw and clean unigram charts separately makes the effect of preprocessing much easier to see. The raw token view still contains more generic language, while the cleaned token view concentrates attention on terms that are easier to connect to substantive inquiry themes.

What the result suggests. Terms such as `political`, `voters`, `aec`, and `vote` indicate that the corpus remains strongly centered on electoral administration, participation, and institutional process even after general context words are filtered out. That is a useful sign that the cleaning strategy is preserving topic relevant vocabulary rather than stripping too much meaning away.

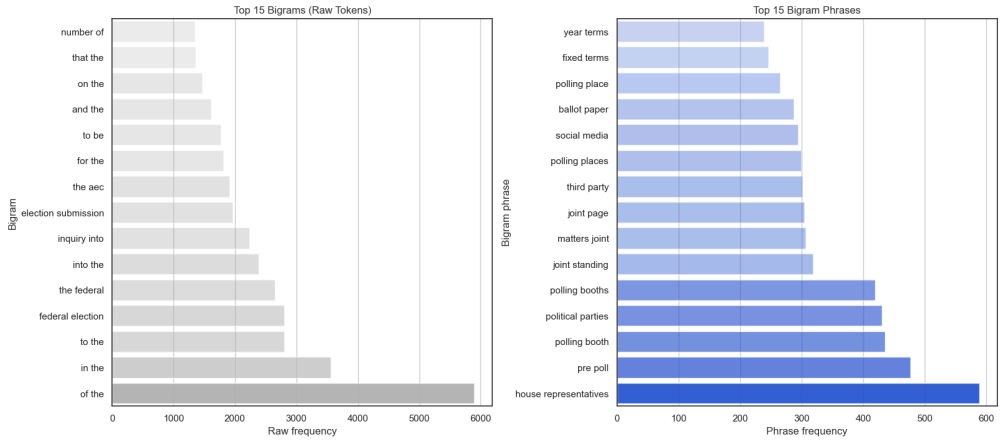
4.6 Bigram phrase visuals (frequency chart and network graph)

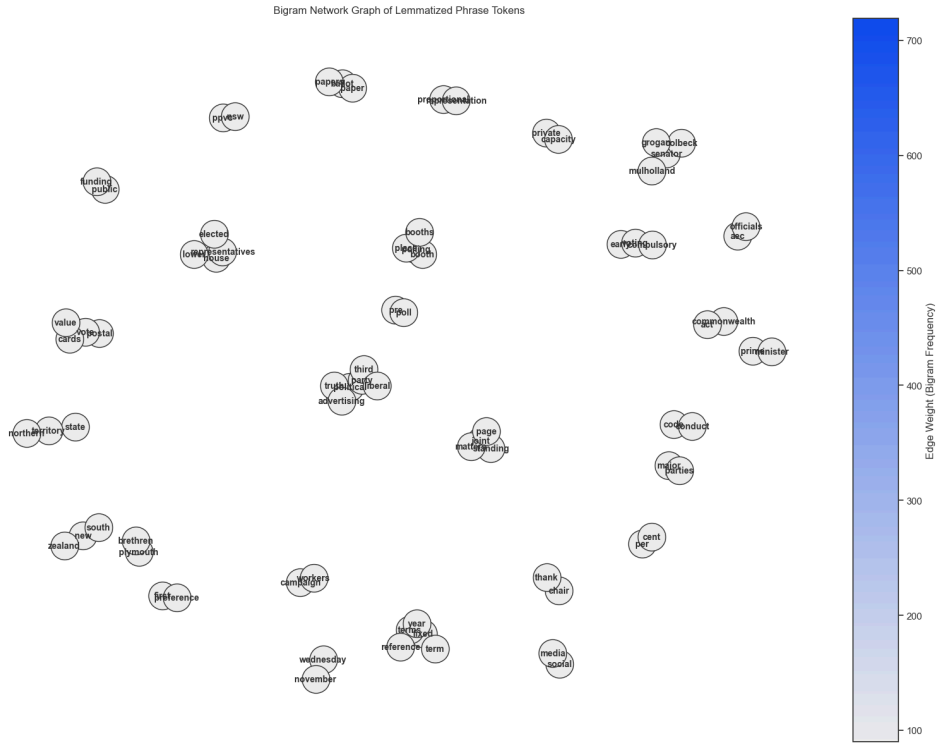
This subsection compares frequent raw bigrams with detected NPMI bigram phrases and then visualizes the phrase based bigram structure as a network using the lemmatized phrase tokens.

The two main ideas here are **collocations** and **networks**. A collocation is a word combination that tends to appear together, while a network graph helps show how those phrase elements connect across the corpus rather than only how often they appear.

In [25]:

```
# 4.6 Bigram phrase visuals

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(16, 7),
constrained_layout=True)

visuals.bar(
    data=top_raw_bigrams.sort_values("count", ascending=True),
    x="count",
    y="bigram",
    color=UTS_PALETTE["mid_grey"],
    title="Top 15 Bigrams (Raw Tokens)",
    xlabel="Raw frequency",
    ylabel="Bigram",
    ax=axes[0],
)

visuals.bar(
    data=top_phrase_bigrams.sort_values("count", ascending=True),
    x="count",
    y="bigram",
    color=UTS_PALETTE["blue"],
    title="Top 15 Bigram Phrases",
    xlabel="Phrase frequency",
    ylabel="Bigram phrase",
    ax=axes[1],
)

plt.show()

lemmatized_phrase_tokens = [
    token
    for tokens in corpus_df["lemmatized_tokens"]
    if isinstance(tokens, list)
    for token in tokens
    if isinstance(token, str) and token.count("_") == 1
]

visuals.bigram_network(
    tokens=lemmatized_phrase_tokens,
    title="Bigram Network Graph of Lemmatized Phrase Tokens",
    n=50,
    figsize=(16, 12),
)
```





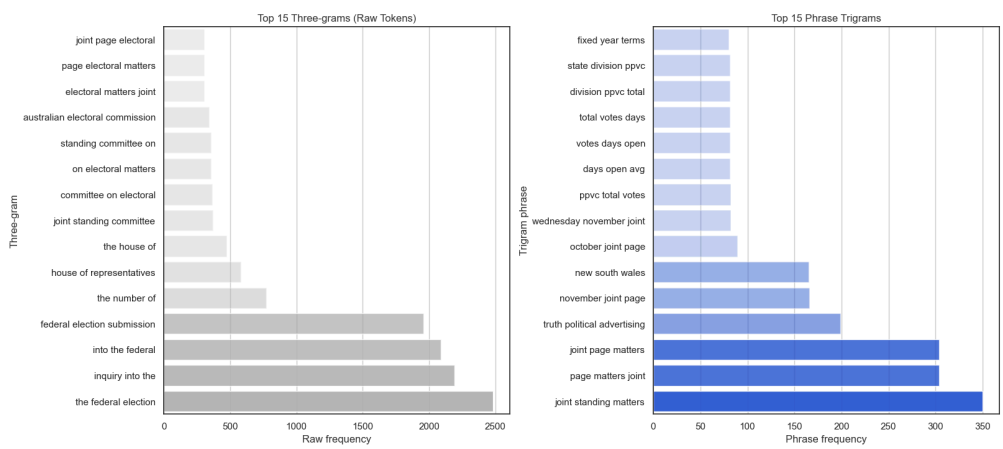
Bigrams add more context than single terms, and the NPMI phrase model highlights word pairs that cohere strongly enough to be treated as phrase like units. Comparing raw bigrams with detected NPMI bigram phrases shows the difference between simple adjacency and more meaningful collocations.

4.7 Top three-gram frequency chart

This section compares raw trigram frequencies with phrase-detected trigrams to show which three-word patterns emerge directly from token sequences versus phrase model detection.

In [26]:

```
# 4.7 Visual: top three grams

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(16, 7),
constrained_layout=True)

visuals.bar(
    data=top_raw_trigrams.sort_values("count", ascending=True),
    x="count",
    y="trigram",
    color=UTS_PALETTE["mid_grey"],
    title="Top 15 Three-grams (Raw Tokens)",
    xlabel="Raw frequency",
    ylabel="Three-gram",
    ax=axes[0],
)

visuals.bar(
    data=top_phrase_trigrams.sort_values("count", ascending=True),
    x="count",
    y="trigram",
    color=UTS_PALETTE["blue"],
    title="Top 15 Phrase Trigrams",
    xlabel="Phrase frequency",
    ylabel="Trigram phrase",
    ax=axes[1],
)

plt.show()
```



The trigram visual now complements the bigram analysis by highlighting higher-order phrase patterns for interpretation.

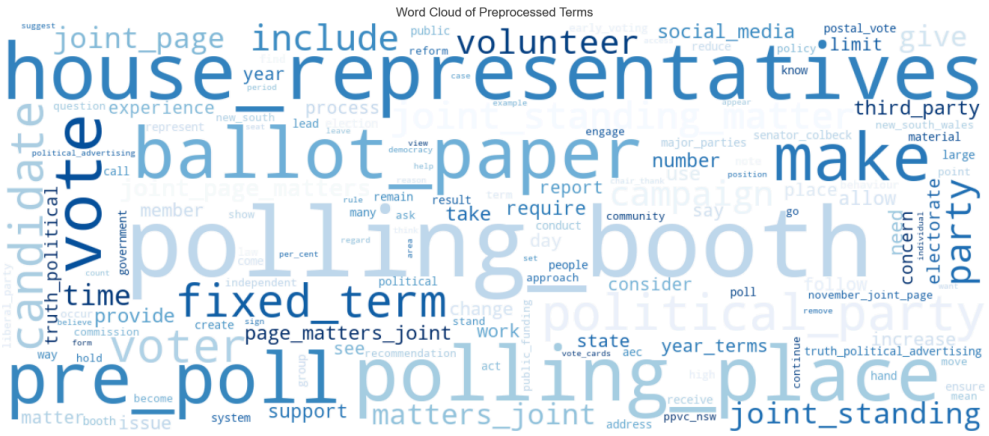
4.8 Word cloud of preprocessed corpus

This visual summary highlights the relative prominence of terms in the fully preprocessed corpus.

A **word cloud** is a communication oriented visualization: larger words appear more prominent, but the chart is approximate rather than exact. It works well as a storytelling device at the end of the preprocessing section because it gives a quick visual impression of the vocabulary before the notebook moves into formal modeling.

In [27]:

```
# 4.8 Word cloud

wordcloud_text = " ".join(corpus_df["preprocessed_text"].fillna(""))

visuals.wordcloud(
    text=wordcloud_text,
    title="Word Cloud of Preprocessed Terms",
    width=1200,
    height=500,
    max_words=150,
    colormap="Blues",
    background_color=UTS_PALETTE["white"],
    figsize=(14, 6),
)
```



The largest terms like *polling_place*, *polling_booth*, *pre_poll*, *ballot_paper*, *house*, *representatives*, *fixed_term* are all NPMI detected bigram phrases, confirming that the phrase detection step successfully elevated multi word concepts above their individual components. The physical infrastructure of elections (*polling_place*, *polling_booth*, *pre_poll*) visually dominates, suggesting that the corpus is heavily concerned with where and how Australians cast their votes. Institutional phrases like *house* (of) *representatives*, *joint_standing*, *matters_joint*, and *joint_page* reflect the parliamentary committee context in which the inquiry operates.

Single token terms fill the surrounding space: *vote*, *candidate*, *voter*, *party*, *political*, *make*, *number*, *time*, *volunteer*, and *campaign*. These map neatly onto the democratic process vocabulary already seen in the bar charts but here appear alongside their phrasal counterparts, reinforcing that electoral participation and candidate selection are the twin pillars of the discourse. Smaller but still visible terms like *social_media*, *third_party*, *political_advertising*, *truth_political_advertising*, *postal_vote*, *vote_card* point to secondary policy debates around campaign regulation, media influence, and alternative voting methods.

5. Advanced Methods (Topic Modeling and Clustering)

This section extends the descriptive analysis with topic modeling and unsupervised clustering so the corpus can be compared through latent themes, document groupings, and broader patterns of policy discussion.

Core concepts introduced here:
- **LDA topic modeling** estimates hidden themes by finding groups of words that tend to appear together across documents.
- **TF IDF** gives more weight to terms that help distinguish one document from another.
- **Clustering** groups documents that are similar in vocabulary or semantic profile.
- **Dimensionality reduction** simplifies high dimensional text features so they can be modeled and visualized more clearly.
- **PCA (Principal Component Analysis)** is a linear dimensionality reduction technique that projects the data onto orthogonal axes of maximum variance, useful as an initial reduction step before clustering or visualization.

5.1 Document Vectorization from Lemmatized Tokens

This section builds three separate document-level feature representations from `corpus_df["lemmatized_tokens"]` :

1. Bag of Words counts (`X_bow_counts`)
2. TF-IDF (`X_tfidf`)
3. Continuous Bag of Words Word2Vec document embeddings (`X_cbow_w2v`)

In [28]:
```
# Prepare corpus text from the existing lemmatized token column
lemmatized_docs = corpus_df["lemmatized_tokens"].tolist()
vectorization_texts = corpus_df["lemmatized_tokens"].apply(" ".join).tolist()

print(f"Documents available for vectorization: {len(vectorization_texts)}")
print(f"Example document text: {vectorization_texts[0][:140]}...")
```

```
Documents available for vectorization: 283
Example document text: time revisit territory representation introduction act senate anomalous arrangement year enrolment increase size state remain unchanged terr...
```

In [29]:
```
# 1) Bag of Words (raw token counts)
bow_vectorizer = CountVectorizer(max_df=0.9,min_df=3)
X_bow_counts = bow_vectorizer.fit_transform(vectorization_texts)
bow_feature_names = bow_vectorizer.get_feature_names_out()

print("BoW matrix shape:", X_bow_counts.shape)
print("BoW vocabulary size:", len(bow_feature_names))
```

```
BoW matrix shape: (283, 8552)
BoW vocabulary size: 8552
```

In [30]:
```
# 2) TF-IDF weighting
tfidf_vectorizer = TfidfVectorizer(max_df=0.9, min_df=3)
X_tfidf = tfidf_vectorizer.fit_transform(vectorization_texts)
tfidf_feature_names = tfidf_vectorizer.get_feature_names_out()

print("TF-IDF matrix shape:", X_tfidf.shape)
print("TF-IDF vocabulary size:", len(tfidf_feature_names))
```

```
TF-IDF matrix shape: (283, 8552)
TF-IDF vocabulary size: 8552
```

In [31]:
```
# 3) Continuous Bag of Words (CBOW) Word2Vec document embeddings
from gensim.models import Word2Vec

cbow_w2v_model = Word2Vec(
    sentences=lemmatized_docs,
    vector_size=100,
    window=5,
    min_count=3,
    workers=1,
    sg=0,  # CBOW architecture
    seed=42,
)

def document_embedding(tokens, model):
    vectors = [model.wv[token] for token in tokens if token in
model.wv]
    if not vectors:
        return np.zeros(model.vector_size, dtype=float)
    return np.mean(vectors, axis=0)

X_cbow_w2v = np.vstack([document_embedding(tokens, cbow_w2v_model)
for tokens in lemmatized_docs])

print("CBOW Word2Vec matrix shape:", X_cbow_w2v.shape)
print("CBOW embedding dimension:", cbow_w2v_model.vector_size)
print("CBOW vocabulary size:", len(cbow_w2v_model.wv))
```

```
Exception ignored in:
'gensim.models.word2vec_inner.our_dot_float'

CBOW Word2Vec matrix shape: (283, 100)
CBOW embedding dimension: 100
CBOW vocabulary size: 13339
```

5.2 LDA Topic Modeling with Bag-of-Words

This cell evaluates candidate topic counts with a coherence score, selects the strongest LDA solution, and summarizes each topic using high probability terms.

What is LDA? Latent Dirichlet Allocation (LDA) is a topic modeling method that treats each document as a mixture of topics and each topic as a mixture of words. It does not assign one fixed label to a document by itself, instead, it estimates probabilities that indicate how strongly each topic appears in each submission.

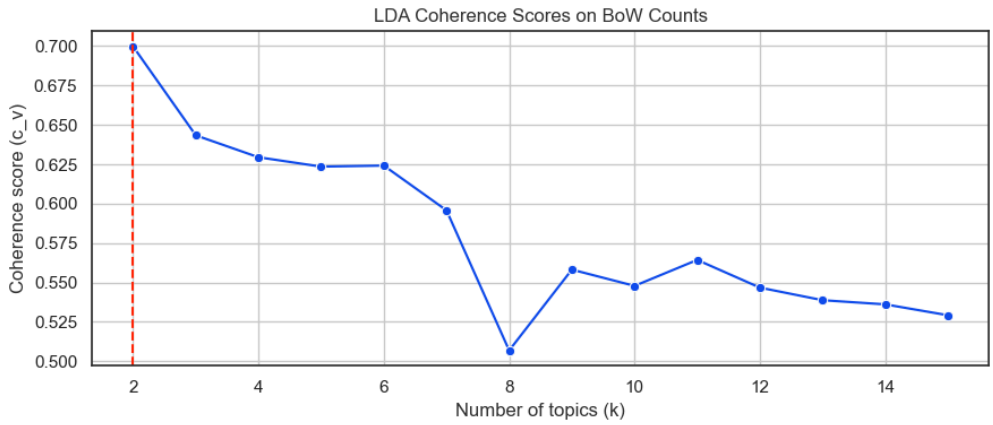
What is coherence? Topic coherence is a quality check that estimates whether the top words in a topic make semantic sense together. In this notebook, coherence is used to compare several candidate topic counts so the final model is selected on an interpretable basis rather than by guesswork.

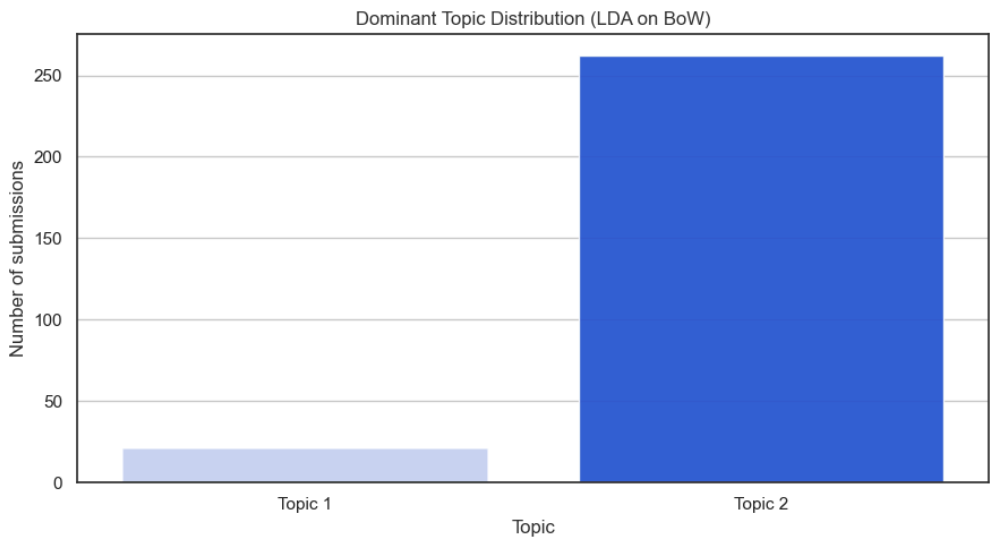
In [ ]:

```
# Reuse the BoW matrix prepared at the start of Section 5
bow_dtm = X_bow_counts
bow_feature_names = bow_vectorizer.get_feature_names_out()

topic_candidates = list(range(2, 16))
coherence_rows = []
lda_candidates = {}

for k in topic_candidates:
    candidate_model = LatentDirichletAllocation(
        n_components=k,
        random_state=42,
        learning_method="batch",
        max_iter=25,
    )
    candidate_model.fit(bow_dtm)
    lda_candidates[k] = candidate_model
    coherence_rows.append(
        {
            "k": k,
            "coherence_score":
TextAnalytics.get_coherence_score(lemmatized_docs, candidate_model,
bow_feature_names),
        }
    )

coherence_df = pd.DataFrame(coherence_rows)
n_topics = int(coherence_df.sort_values(["coherence_score", "k"],
ascending=[False, True]).iloc[0]["k"])
lda_model = lda_candidates[n_topics]
topic_distributions = lda_model.transform(bow_dtm)

corpus_df["dominant_topic"] = topic_distributions.argmax(axis=1) + 1
topic_terms_df =
TextAnalytics.extract_top_terms_per_topic(lda_model,
bow_feature_names, top_n=10)

LOGGER.debug(
    "LDA fitted on BoW counts: topics=%d, vocab=%d, coherence=%.4f",
    n_topics,
    len(bow_feature_names),
    coherence_df.loc[coherence_df["k"] == n_topics,
"coherence_score"].iloc[0],
)

plt.figure(figsize=(9, 4))
sns.lineplot(data=coherence_df, x="k", y="coherence_score",
marker="o", color=UTS_PALETTE["blue"])
plt.axvline(n_topics, color=UTS_PALETTE["red"], linestyle="--",
linewidth=1.5)
plt.title("LDA Coherence Scores on BoW Counts")
plt.xlabel("Number of topics (k)")
plt.ylabel("Coherence score (c_v)")
```

```
plt.tight_layout()
plt.show()

topic_counts = (
    corpus_df["dominant_topic"]
    .value_counts()
    .sort_index()
    .rename_axis("topic")
    .reset_index(name="count")
)

topic_counts["topic"] =
topic_counts["topic"].astype(str).radd("Topic ")
visuals.bar(
    data=topic_counts,
    x="topic",
    y="count",
    color=UTS_PALETTE["blue"],
    title="Dominant Topic Distribution (LDA on BoW)",
    xlabel="Topic",
    ylabel="Number of submissions",
    figsize=(9, 5),
)

display(coherence_df)
display(topic_terms_df)

# Faceted bar chart: top-weighted terms per LDA topic
lda_top_n = 10
topic_term_rows = []
for topic_idx, topic_weights in enumerate(lda_model.components_):
    top_indices = topic_weights.argsort()[::-1][:lda_top_n]
    for rank, idx in enumerate(top_indices):
        topic_term_rows.append({
            "topic": f"Topic {topic_idx + 1}",
            "term": bow_feature_names[idx],
            "weight": topic_weights[idx],
        })

lda_term_weights_df = pd.DataFrame(topic_term_rows)
lda_topics_list = sorted(lda_term_weights_df["topic"].unique())

fig, axes = plt.subplots(
    nrows=len(lda_topics_list), ncols=1,
    figsize=(11, max(3.2 * len(lda_topics_list), 6)),
    constrained_layout=True,
)
if len(lda_topics_list) == 1:
    axes = [axes]

for ax, topic_label in zip(axes, lda_topics_list):
    topic_data = (
        lda_term_weights_df[lda_term_weights_df["topic"] ==
```

```
topic_label]
        .sort_values("weight", ascending=True)
    )
    visuals.bar(
        data=topic_data,
        x="weight",
        y="term",
        color=UTS_PALETTE["blue"],
        title=f"Top {lda_top_n} Terms — {topic_label}",
        xlabel="LDA weight",
        ylabel="Term",
        ax=ax,
    )

plt.show()
```

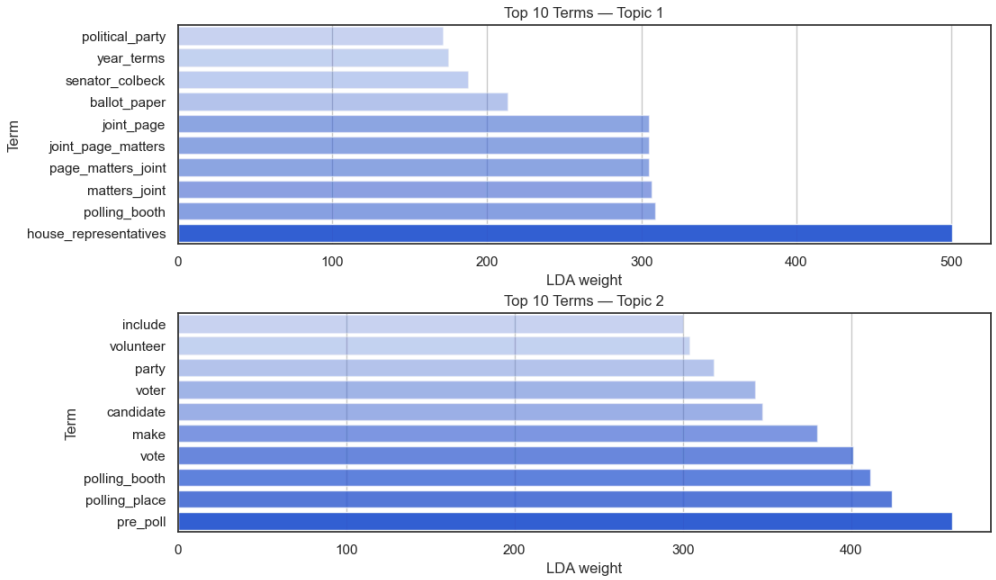




	k	coherence_score
0	2	0.699779
1	3	0.643376
2	4	0.629467

	k	coherence_score
3	5	0.623522
4	6	0.624169
5	7	0.595756
6	8	0.506835
7	9	0.558142
8	10	0.547813
9	11	0.564283
10	12	0.546716
11	13	0.538728
12	14	0.536083
13	15	0.529180

	topic	top_terms
0	Topic 1	house_representatives, polling_booth, matters_...
1	Topic 2	pre_poll, polling_place, polling_booth, vote, ...



Coherence curve. The coherence plot peaks at **k = 2** (c_v ≈ 0.70), then declines steadily.

Topic distribution. The dominant-topic bar chart reveals a heavily imbalanced split: **Topic 2 absorbs the vast majority of submissions (~260)**, while **Topic 1 captures only ~20 documents**.

Topic 1: Parliamentary procedure and institutional structure. The top terms are *house_representatives*, *polling_booth*, *matters_joint*, *page_matters_joint*, *joint_page_matters*, *joint_page*, *ballot_paper*, *senator_colbeck*, *year_terms*, and *political_party*. These are heavily procedural and institutional: references to the House of Representatives, joint committee page metadata, specific senators, and fixed parliamentary terms.

Topic 2: Electoral operations and voter participation. The top terms are *pre_poll*, *polling_place*, *polling_booth*, *vote*, *make*, *candidate*, *voter*, *party*, *volunteer*, and *include*. This is the dominant topic and it reads as a coherent policy discussion about the practical conduct of elections.

5.3 TF-IDF Clustering Experiment (Hierarchical + K-means)

This section applies hierarchical clustering and K-means on TF-IDF features to identify document groupings and compare cluster structure across methods.

In [ ]:
```
def top_tokens_from_cluster_assignments(token_docs, labels,
top_n=12):
    rows = []
    for cluster_id in sorted(np.unique(labels)):
        doc_indices = np.where(labels == cluster_id)[0]
        token_counter = Counter(token for doc_idx in doc_indices for
token in token_docs[doc_idx])
        rows.append(
            {
                "cluster": f"Cluster {cluster_id + 1}",
                "n_docs": len(doc_indices),
                "top_terms": ", ".join([token for token, _ in
token_counter.most_common(top_n)]),
            }
        )
    return pd.DataFrame(rows)

def source_breakdown_tfidf(matrix, labels, feature_names,
evidence_styles, top_n=8):
    """Top terms per cluster split by Submission vs Hansard, TF-IDF
weighted."""
    matrix_dense = matrix.toarray() if hasattr(matrix, "toarray")
else np.asarray(matrix)
    labels_arr = np.asarray(labels)
    styles_arr = np.asarray(evidence_styles)
    breakdown = {}
    for cid in sorted(np.unique(labels_arr)):
        mask = labels_arr == cid
        info = {}
        for style, key in [("Submission", "submission"), ("Hansard",
"hansard")]:
            smask = mask & (styles_arr == style)
            n = int(smask.sum())
            info[f"n_{key}"] = n
            if n > 0:
                mean_w = matrix_dense[smask].mean(axis=0)
                top_idx = np.argsort(mean_w)[::-1][:top_n]
                info[f"{key}_terms"] = ",
".join(feature_names[top_idx])
            else:
                info[f"{key}_terms"] = ""
        breakdown[f"Cluster {int(cid) + 1}"] = info
    return breakdown

def source_breakdown_tokens(token_docs, labels, evidence_styles,
top_n=8):
    """Top tokens per cluster split by Submission vs Hansard."""
    labels_arr = np.asarray(labels)
    styles_arr = np.asarray(evidence_styles)
    breakdown = {}
    for cid in sorted(np.unique(labels_arr)):
        mask = labels_arr == cid
        info = {}
```

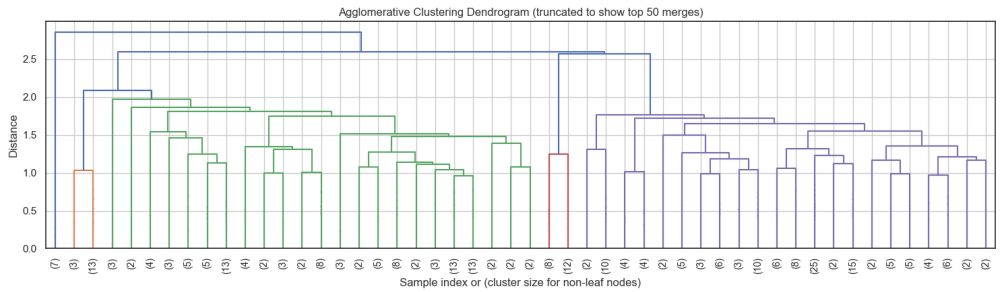
```
        for style, key in [("Submission", "submission"), ("Hansard",
"hansard")]:
            smask = mask & (styles_arr == style)
            n = int(smask.sum())
            info[f"n_{key}"] = n
            if n > 0:
                indices = np.where(smask)[0]
                counter = Counter(t for i in indices for t in
token_docs[i])
                info[f"{key}_terms"] = ", ".join([t for t, _ in
counter.most_common(top_n)])
            else:
                info[f"{key}_terms"] = ""
        breakdown[f"Cluster {int(cid) + 1}"] = info
    return breakdown

def print_cluster_summary(title, terms_df):
    print(f"\n{title}")
    has_n_docs = "n_docs" in terms_df.columns
    for _, row in terms_df.sort_values("cluster").iterrows():
        if has_n_docs and pd.notna(row["n_docs"]):
            print(f"- {row['cluster']} ({int(row['n_docs'])} docs)")
        else:
            print(f"- {row['cluster']}")
        print(f"  {row['top_terms']}")

def plot_cluster_term_cards(title, terms_df, n_cols=2,
source_breakdown=None):
    terms_df =
terms_df.sort_values("cluster").reset_index(drop=True)
    has_n_docs = "n_docs" in terms_df.columns
    has_source = source_breakdown is not None
    n_items = len(terms_df)
    n_rows = int(np.ceil(n_items / n_cols))
    row_h = 4.5 if has_source else 2.9

    fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, row_h *
n_rows))
    axes = np.atleast_1d(axes).flatten()

    wrap_w = 52
    src_wrap_w = max(30, int(48 * 2 / max(n_cols, 1)))

    for ax, (_, row) in zip(axes, terms_df.iterrows()):
        wrapped_terms = textwrap.fill(row["top_terms"],
width=wrap_w)
        card_title = str(row["cluster"])
        if has_n_docs and pd.notna(row["n_docs"]):
            card_title = f"{card_title}  ({int(row['n_docs'])}
docs)"
        ax.text(
            0.02, 0.96, card_title,
            transform=ax.transAxes, fontsize=11, fontweight="bold",
```

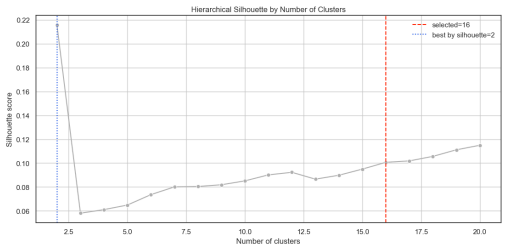
```
            va="top", color=UTS_PALETTE["blue"],
        )
        ax.text(0.02, 0.87, wrapped_terms, transform=ax.transAxes,
fontsize=10, va="top")

        if has_source:
            info = source_breakdown.get(str(row["cluster"]), {})
            sub_n = info.get("n_submission", 0)
            sub_terms = info.get("submission_terms", "")
            han_n = info.get("n_hansard", 0)
            han_terms = info.get("hansard_terms", "")

            sub_text = textwrap.fill(
                f"Submission ({sub_n}): {sub_terms}" if sub_terms
else f"Submission ({sub_n}): \u2014",
                width=src_wrap_w,
            )
            han_text = textwrap.fill(
                f"Hansard ({han_n}): {han_terms}" if han_terms else
f"Hansard ({han_n}): \u2014",
                width=src_wrap_w,
            )
            ax.text(0.02, 0.42, sub_text, transform=ax.transAxes,
                    fontsize=8.5, va="top", color="#2E86AB",
fontstyle="italic")
            ax.text(0.02, 0.20, han_text, transform=ax.transAxes,
                    fontsize=8.5, va="top", color="#A23B72",
fontstyle="italic")

        ax.axis("off")

    for ax in axes[n_items:]:
        ax.axis("off")

    fig.suptitle(title, fontsize=13, fontweight="bold", y=1.01)
    plt.tight_layout()
    plt.show()

# Use the TF-IDF matrix already created in Section 5 vectorization
n_tfidf_components = max(2, min(50, X_tfidf.shape[0] - 1,
X_tfidf.shape[1] - 1))
tfidf_svd = TruncatedSVD(n_components=n_tfidf_components,
random_state=42)
X_tfidf_reduced = tfidf_svd.fit_transform(X_tfidf)

# Evaluate number of clusters (cluster_num) up to 20, select 16 as
the final value
tfidf_hier_cluster_num_candidates = list(range(2, 21))
tfidf_hier_eval = []
for cluster_num in tfidf_hier_cluster_num_candidates:
    labels = AgglomerativeClustering(n_clusters=cluster_num,
linkage="ward").fit_predict(X_tfidf_reduced)
    tfidf_hier_eval.append(
```

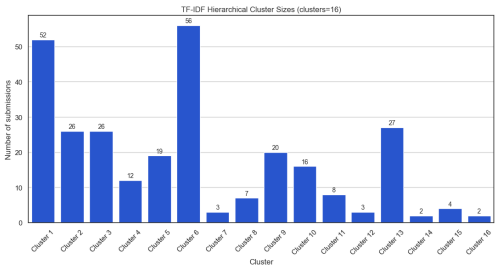
```
        {
            "cluster_num": cluster_num,
            "silhouette_score": silhouette_score(X_tfidf_reduced,
labels),
        }
    )

tfidf_hier_eval_df = pd.DataFrame(tfidf_hier_eval)
tfidf_hier_auto_best_cluster_num =
int(tfidf_hier_eval_df.loc[tfidf_hier_eval_df["silhouette_score"].id
xmax(), "cluster_num"])
tfidf_hier_selected_cluster_num = 16  # FINAL NUMBER OF CLUSTERS

tfidf_hier_labels =
AgglomerativeClustering(n_clusters=tfidf_hier_selected_cluster_num,
linkage="ward").fit_predict(X_tfidf_reduced)
corpus_df["tfidf_hierarchical_cluster"] = tfidf_hier_labels + 1

tfidf_hier_counts = (
    corpus_df["tfidf_hierarchical_cluster"]
    .value_counts()
    .sort_index()
    .rename_axis("cluster")
    .reset_index(name="count")
)
tfidf_hier_counts["cluster"] =
tfidf_hier_counts["cluster"].astype(str).radd("Cluster ")

tfidf_hier_top_terms_tfidf =
TextAnalytics.extract_top_terms_per_cluster(
    X_tfidf,
    tfidf_hier_labels,
    tfidf_feature_names,
    top_n=10,
)

tfidf_hier_top_terms_tokens = top_tokens_from_cluster_assignments(
    lemmatized_docs,
    tfidf_hier_labels,
    top_n=12,
)

LOGGER.debug(
    "TF-IDF hierarchical clustering complete: docs=%d, features=%d,
clusters=%d",
    len(corpus_df),
    len(tfidf_feature_names),
    tfidf_hier_selected_cluster_num,
)

# Dendrogram plot
plt.figure(figsize=(15, 4.5))
plt.title("Agglomerative Clustering Dendrogram (truncated to show
```

```
top 50 merges)")
Z = linkage(X_tfidf_reduced, method='ward')
dendrogram(
    Z,
    truncate_mode="lastp",
    p=50,
    leaf_rotation=90.,
    leaf_font_size=10.,
    show_contracted=True,
)
plt.xlabel("Sample index or (cluster size for non-leaf nodes)")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()

# Plots for silhouette and cluster sizes (WIDE for cluster label
clarity)
fig, axes = plt.subplots(1, 2, figsize=(22, 6))  # Make wider to see
cluster labels

sns.lineplot(
    data=tfidf_hier_eval_df,
    x="cluster_num",
    y="silhouette_score",
    marker="o",
    ax=axes[0],
    color=UTS_PALETTE["mid_grey"],
)
axes[0].axvline(tfidf_hier_selected_cluster_num,
color=UTS_PALETTE["red"], linestyle="--", label=f"selected=
{tfidf_hier_selected_cluster_num}")
axes[0].axvline(
    tfidf_hier_auto_best_cluster_num,
    color=UTS_PALETTE["blue"],
    linestyle=":",
    label=f"best by silhouette={tfidf_hier_auto_best_cluster_num}",
)
axes[0].set_title("Hierarchical Silhouette by Number of Clusters")
axes[0].set_xlabel("Number of clusters")
axes[0].set_ylabel("Silhouette score")
axes[0].legend(frameon=False)

sns.barplot(data=tfidf_hier_counts, x="cluster", y="count",
color=UTS_PALETTE["blue"], ax=axes[1])
axes[1].set_title(f"TF-IDF Hierarchical Cluster Sizes (clusters=
{tfidf_hier_selected_cluster_num})")
axes[1].set_xlabel("Cluster")
axes[1].set_ylabel("Number of submissions")
for container in axes[1].containers:
    axes[1].bar_label(container, padding=2, fontsize=10)
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
```

```
plt.show()

tfidf_hier_source_bkdn = source_breakdown_tfidf(
    X_tfidf, tfidf_hier_labels, tfidf_feature_names,
    corpus_df["evidence_style"].values, top_n=8,
)
plot_cluster_term_cards(
    f"Hierarchical clusters ({tfidf_hier_selected_cluster_num}): top
TF-IDF weighted terms",
    tfidf_hier_top_terms_tfidf,
    n_cols=4,
    source_breakdown=tfidf_hier_source_bkdn,
)
```







Hierarchical clusters (16): top TF-IDF weighted terms

Cluster 1 (52 docs)

ballot_papers, vote, state, year, make, political_party, supplementary, year_terms, need, elect

Submission (52): ballot_papers, vote, state, year, make, political_party, supplementary, year_terms

Hansard (0): —

Cluster 10 (16 docs)

house_representatives, year_terms, proportional_representation, fixed_terms, per_cent, fixed_year, section_constitution, prime_minister, fixed_year_terms, increase

Submission (16): house_representatives, year_terms, proportional_representation, fixed_terms, per_cent, fixed_year, section_constitution, prime_minister

Hansard (0): —

Cluster 11 (8 docs)

jewish_community, proxy_groups, third_party_proxy, third_party, repeal_teals, party_proxy_groups, postal_vote, jew, polling_booth, pre_poll

Submission (8): jewish_community, proxy_groups, third_party_proxy, third_party, repeal_teals, party_proxy_groups, postal_vote, jew

Hansard (0): —

Cluster 12 (3 docs)

linear_ranking_system, preferential_linear_ranking, vote_value, proportional_representation, ranking_system, preferential_linear, ranking_system_plers, hare_clark, first_preference, vacancy

Submission (3): linear_ranking_system, preferential_linear_ranking, vote_value, proportional_representation, ranking_system, preferential_linear, ranking_system_plers, hare_clark

Hansard (0): —

Cluster 13 (27 docs)

polling_place, early_voting, campaign_workers, volunteer, pre_poll, voter, vote, aec_officials, make, candidate

Submission (27): polling_place, early_voting, campaign_workers, volunteer, pre_poll, voter, vote, aec_officials

Hansard (0): —

Cluster 14 (2 docs)

engaged_influencers, grattan_institute, senator_james_mcgrath, senator_james, public_hearing_october, mcgrath, forward, influencers, secretariat, partnership

Submission (2): engaged_influencers, grattan_institute, senator_james_mcgrath, senator_james, public_hearing_october, mcgrath, forward, influencers

Hansard (0): —

Cluster 15 (4 docs)

public_funding, major_parties, administrative_funding, postal_vote, new_entrants, political_party, nominated_entity, expenditure_caps, donation_caps, nominated_entities

Submission (4): public_funding, major_parties, administrative_funding, postal_vote, new_entrants, political_party, nominated_entity, expenditure_caps

Hansard (0): —

Cluster 16 (2 docs)

bundle, audit, count, seem, polling_place, gate, worker, first_preferences_counted, entry_exit, witness

Submission (2): bundle, audit, count, seem, polling_place, gate, worker, first_preferences_counted

Hansard (0): —

Cluster 2 (26 docs)

candidate, reps_subject, date_tuesday, polling_booth, returning_officer, volunteer, robert, vote_cards, vote, booth

Submission (26): candidate, reps_subject, date_tuesday, polling_booth, returning_officer, volunteer, robert, vote_cards

Hansard (0): —

Cluster 3 (26 docs)

accessed_october, political_advertising, social_media, truth_political, truth_political_advertising, postal_vote, joint_standing_matters, require, third_party, joint_standing

Submission (26): accessed_october, political_advertising, social_media, truth_political, truth_political_advertising, postal_vote, joint_standing_matters, require

Hansard (0): —

Cluster 4 (12 docs)

pre_poll, pre_poll_voting, volunteer, brighton_pre_poll, pre_poll_booth, pre_poll_ordinary, htv_cards, vote, pre_poll_day, booth_captain

Submission (12): pre_poll, pre_poll_voting, volunteer, brighton_pre_poll, pre_poll_booth, pre_poll_ordinary, htv_cards, vote

Hansard (0): —

Cluster 5 (19 docs)

house_canberra_act, matters_box_house, standing_matters_box, canberra_act, box_house_canberra, standing_matters, joint_standing_matters, secretary_joint_standing, secretary_joint, box

Submission (19): house_canberra_act, matters_box_house, standing_matters_box, canberra_act, box_house_canberra, standing_matters, joint_standing_matters, secretary_joint_standing

Hansard (0): —

Cluster 6 (56 docs)

polling_booth, volunteer, pre_poll, stand, candidate, voter, booth, vote, party, campaign

Submission (56): polling_booth, volunteer, pre_poll, stand, candidate, voter, booth, vote

Hansard (0): —

Cluster 7 (3 docs)

ballot_paper, senate_voting_system, district_magnitude, first_preference, genuine_democratic, malcolm_mackerras, house_representatives, counted_formal, rig, senate_ballot_paper

Submission (3): ballot_paper, senate_voting_system, district_magnitude, first_preference, genuine_democratic, malcolm_mackerras, house_representatives, counted_formal

Hansard (0): —

Cluster 8 (7 docs)

page_matters_joint, joint_page, joint_page_matters, matters_joint, senator_colbeck, november_joint_page, senator_mulholland, polling_booth, senator_grogan, chair_thank

Submission (0): —

Hansard (7): page_matters_joint, joint_page, joint_page_matters, matters_joint, senator_colbeck, november_joint_page, senator_mulholland, polling_booth

Cluster 9 (20 docs)

polling_place, pre_poll, polling_booth, term_reference, aec_officials, terms_reference, early_voting, behaviour_voters_political, volunteer, obstruction_intimidating_behaviour

Submission (20): polling_place, pre_poll, polling_booth, term_reference, aec_officials, terms_reference, early_voting, behaviour_voters_political

Hansard (0): —

Cluster selection rationale.
The silhouette curve peaks at k = 2 (≈ 0.22), indicating that a coarse two way split maximises average intra cluster cohesion. However, a two cluster solution is too coarse for thematic interpretation of a parliamentary inquiry corpus. As k increases the silhouette score declines and flattens around k = 10–16, reflecting the natural overlap among submissions that share common electoral vocabulary. We select **k = 16** to preserve fine grained topical distinctions at this granularity the algorithm separates clearly differentiated policy sub-themes even though overall silhouette (≈ 0.08) is modest. Low silhouette is expected with high-dimensional sparse TF-IDF vectors and Ward linkage on SVD-reduced features.

Dendrogram structure.
The truncated dendrogram reveals an asymmetric merge tree. Two major branches emerge at the highest distance level (≈ 2.8): one groups the larger, more general electoral clusters (left branch), while the other captures specialised and smaller thematic clusters (right branch). Several early low distance merges on the right side correspond to tightly focused micro topics (e.g., voting system reform proposals, audit procedures), whereas the left branch merges occur at higher distances, reflecting greater internal heterogeneity in broad electoral clusters.

Thematic interpretation of selected clusters.

- **Cluster 3 (26 docs)** — *Political advertising and truth in political advertising.* Top terms include `truth_political_advertising`, `social_media`, `accessed_october`, and `third_party`.
- **Cluster 4 (12 docs)** — *Pre-poll voting operations.* Dominated by `pre_poll`, `pre_poll_voting`, `brighton_pre_poll`, and `htv_cards`.
- **Cluster 10 (16 docs)** — *Fixed terms and proportional representation.* Terms such as `house_representatives`, `fixed_terms`, `proportional_representation`, `section_constitution`, and `prime_minister`.
- **Cluster 11 (8 docs)** — *Proxy voting and community-specific concerns.* The distinctive terms `jewish_community`, `proxy_groups`, and `third_party_proxy`.
- **Cluster 12 (3 docs)** — *Alternative ranking systems.* `linear_ranking_system`, `preferential_linear_ranking`, and `hare_clark`.
- **Cluster 15 (4 docs)** — *Political funding and expenditure caps.* `public_funding`, `donation_caps`, `expenditure_caps`, and `nominated_entity`

Source-type breakdown.
Almost all clusters draw exclusively from written submissions, with zero Hansard transcripts. The notable exception is **Cluster 8 (7 docs)**, which is composed entirely of Hansard records (`page_matters_joint`, `senator_colbeck`, `senator_mulholland`, `chair_thank`), capturing parliamentary committee hearing discourse rather than public submissions.

Limitations of this partition.

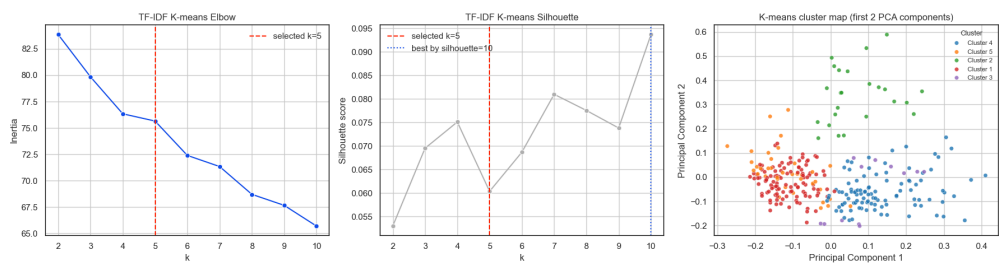
The low overall silhouette score reflects substantial vocabulary overlap, many submissions discuss polling, volunteers, and voting in similar terms regardless of their primary policy focus. Clusters 1 and 6 act as residual "catch-all" groups, absorbing documents that do not contain sufficiently distinctive n-grams.

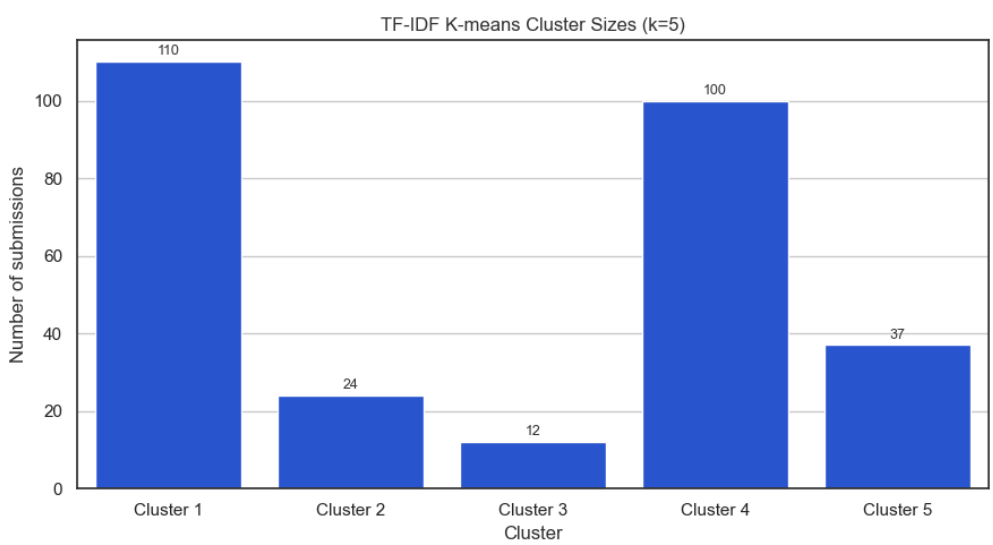
In [34]:

```
# 5.3B TF-IDF K-means clustering

from sklearn.decomposition import PCA

# Evaluate candidate k values up to 10, but force the selected value
to k=5
k_candidates = list(range(2, 11))
tfidf_kmeans_eval = []
tfidf_kmeans_label_map = {}

for k in k_candidates:
    model = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = model.fit_predict(X_tfidf_reduced)
    tfidf_kmeans_label_map[k] = labels
    tfidf_kmeans_eval.append(
        {
            "k": k,
            "inertia": model.inertia_,
            "silhouette_score": silhouette_score(X_tfidf_reduced,
labels),
        }
    )

tfidf_kmeans_eval_df = pd.DataFrame(tfidf_kmeans_eval)
tfidf_auto_best_k =
int(tfidf_kmeans_eval_df.loc[tfidf_kmeans_eval_df["silhouette_score"
].idxmax(), "k"])
tfidf_best_k = 5

if tfidf_best_k not in tfidf_kmeans_label_map:
    raise ValueError("Selected tfidf_best_k is not in evaluated
k_candidates")

tfidf_kmeans_labels = tfidf_kmeans_label_map[tfidf_best_k]
corpus_df["tfidf_kmeans_cluster"] = tfidf_kmeans_labels + 1

tfidf_kmeans_counts = (
    corpus_df["tfidf_kmeans_cluster"]
    .value_counts()
    .sort_index()
    .rename_axis("cluster")
    .reset_index(name="count")
)
tfidf_kmeans_counts["cluster"] =
tfidf_kmeans_counts["cluster"].astype(str).radd("Cluster ")

tfidf_kmeans_top_terms_tfidf =
TextAnalytics.extract_top_terms_per_cluster(
    X_tfidf,
    tfidf_kmeans_labels,
    tfidf_feature_names,
    top_n=10,
)
```

```
tfidf_kmeans_top_terms_tokens = top_tokens_from_cluster_assignments(
    lemmatized_docs,
    tfidf_kmeans_labels,
    top_n=12,
)

# 2D map for visual intuition of cluster separation - use PCA
instead of SVD slice
pca = PCA(n_components=2, random_state=42)
X_tfidf_pca = pca.fit_transform(X_tfidf)

tfidf_plot_df = pd.DataFrame(X_tfidf_pca, columns=["PC1", "PC2"])
tfidf_plot_df["cluster"] =
corpus_df["tfidf_kmeans_cluster"].astype(str).radd("Cluster ")

fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))

sns.lineplot(data=tfidf_kmeans_eval_df, x="k", y="inertia",
marker="o", ax=axes[0], color=UTS_PALETTE["blue"])
axes[0].axvline(tfidf_best_k, color=UTS_PALETTE["red"], linestyle="-
-", label="selected k=5")
axes[0].set_title("TF-IDF K-means Elbow")
axes[0].set_xlabel("k")
axes[0].set_ylabel("Inertia")
axes[0].legend(frameon=False)

sns.lineplot(data=tfidf_kmeans_eval_df, x="k", y="silhouette_score",
marker="o", ax=axes[1], color=UTS_PALETTE["mid_grey"])
axes[1].axvline(tfidf_best_k, color=UTS_PALETTE["red"], linestyle="-
-", label="selected k=5")
axes[1].axvline(tfidf_auto_best_k, color=UTS_PALETTE["blue"],
linestyle=":", label=f"best by silhouette={tfidf_auto_best_k}")
axes[1].set_title("TF-IDF K-means Silhouette")
axes[1].set_xlabel("k")
axes[1].set_ylabel("Silhouette score")
axes[1].legend(frameon=False)

sns.scatterplot(
    data=tfidf_plot_df,
    x="PC1",
    y="PC2",
    hue="cluster",
    palette="tab10",
    s=28,
    alpha=0.8,
    ax=axes[2],
)
axes[2].set_title("K-means cluster map (first 2 PCA components)")
axes[2].set_xlabel("Principal Component 1")
axes[2].set_ylabel("Principal Component 2")
axes[2].legend(title="Cluster", fontsize=8, title_fontsize=9,
frameon=False, loc="best")
```

```
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(9, 5))
sns.barplot(data=tfidf_kmeans_counts, x="cluster", y="count",
color=UTS_PALETTE["blue"], ax=ax)
ax.set_title("TF-IDF K-means Cluster Sizes (k=5)")
ax.set_xlabel("Cluster")
ax.set_ylabel("Number of submissions")
for container in ax.containers:
    ax.bar_label(container, padding=2, fontsize=9)
plt.tight_layout()
plt.show()

tfidf_kmeans_source_bkdn = source_breakdown_tfidf(
    X_tfidf, tfidf_kmeans_labels, tfidf_feature_names,
    corpus_df["evidence_style"].values, top_n=8,
)
plot_cluster_term_cards(
    "K-means clusters: top TF-IDF weighted terms",
    tfidf_kmeans_top_terms_tfidf,
    source_breakdown=tfidf_kmeans_source_bkdn,
)
```





K-means clusters: top TF-IDF weighted terms

Cluster 1 (110 docs)

polling_booth, volunteer, candidate, vote, booth, voter, polling_place, make, experience, hand

Submission (110): polling_booth, volunteer, candidate, vote, booth, voter, polling_place, make

Hansard (0): —

Cluster 2 (24 docs)

polling_place, pre_poll, polling_booth, term_reference, aec_officials, terms_reference, early_voting, obstruction_intimidating_behaviour, deliberate_obstruction_intimidating, behaviour_voters_political

Submission (24): polling_place, pre_poll, polling_booth, term_reference, aec_officials, terms_reference, early_voting, obstruction_intimidating_behaviour

Hansard (0): —

Cluster 3 (12 docs)

joint_page_matters, page_matters_joint, joint_page, matters_joint, senator_colbeck, november_joint_page, preferential_linear_ranking, linear_ranking_system, engaged_influencers, senator_mulholland

Submission (5): linear_ranking_system, preferential_linear_ranking, engaged_influencers, vote_value, grattan_institute, proportional_representation, ranking_system, preferential_linear

Hansard (7): page_matters_joint, joint_page, joint_page_matters, matters_joint, senator_colbeck, november_joint_page, senator_mulholland, polling_booth

Cluster 4 (100 docs)

house_representatives, political_party, year_terms, increase, require, ballot_paper, ballot_papers, polling_place, make, change

Submission (100): house_representatives, political_party, year_terms, increase, require, ballot_paper, ballot_papers, polling_place

Hansard (0): —

Cluster 5 (37 docs)

pre_poll, volunteer, house_canberra_act, canberra_act, box_house_canberra, matters_box_house, standing_matters_box, joint_standing_matters, standing_matters, vote

Submission (37): pre_poll, volunteer, house_canberra_act, canberra_act, box_house_canberra, matters_box_house, standing_matters_box, joint_standing_matters

Hansard (0): —

The elbow plot shows a clear bend around k = 4–5, after which inertia decreases more gradually, supporting k = 5 as a reasonable choice from a diminishing returns perspective. The silhouette plot, however, reveals that k = 5 (≈ 0.075) sits near a local trough, while the auto best k = 10 reaches ≈ 0.095. Selecting k = 5 therefore prioritises **interpretability** fewer, broader clusters that are easier to label thematically over maximum statistical separation.

PCA scatter plot.
The two-component PCA projection shows **substantial overlap** among the five clusters, with most documents concentrated in a central cloud between PC1 ∈ [−0.1, 0.2] and PC2 ∈ [−0.1, 0.2]. No clean spatial boundaries are visible, confirming that the clusters represent **soft thematic tendencies** rather than hard-edged separations. The clearest outliers extend along PC1 for Clusters 4 and 5, suggesting these groups contain documents with more distinctive term profiles.

Cluster size distribution.
The partition is strongly imbalanced: **Clusters 1 (110) and 4 (100)** together absorb 74 % of the corpus and act as broad catc all groups, while **Clusters 2 (24), 3 (12), and 5 (37)** capture more focused thematic niches.

Thematic interpretation of each cluster.

- **Cluster 1 (110 docs) — Polling operations and volunteer experience.** Top TF-IDF terms: *polling_booth, volunteer, candidate, vote, booth, voter, polling_place, make, experience, hand*. The vocabulary centres on people who *worked at* or *attended* polling places, with *volunteer*, *experience*, and *hand* signalling first hand, grassroots accounts.

- **Cluster 2 (24 docs) — AEC conduct and polling place incidents.** Top terms: *polling_place, pre_poll, polling_booth, term_reference, aec_officials, terms_reference, early_voting, obstruction_intimidating, deliberate_obstruction_intimidating, behaviour_voters_political*.

- **Cluster 3 (12 docs) — Parliamentary proceedings and voting-system reform.** Top terms: *joint_page_matters, page_matters_joint, joint_page, matters_joint, senator_colbeck, november_joint_page, preferential_linear_ranking, linear_ranking_system, engaged_influencers, senator_mulholland*. Two sub themes converge: (a) Hansard committee metadata (*joint_page_matters*, *senator_colbeck*, *senator_mulholland*) and (b) alternative ranking system proposals (*preferential_linear_ranking*, *linear_ranking_system*).

- **Cluster 4 (100 docs) — Policy reform and institutional electoral framework.** Top terms: *house_representatives, political_party, year_terms, increase, require, ballot_paper, ballot_papers, polling_place, make, change*. Unlike Cluster 1's experiential register, Cluster 4 centres on institutional structures (*house_representatives*, *political_party*).

- **Cluster 5 (37 docs) — Pre-poll voting and ACT-specific concerns.** Top terms: *pre_poll, volunteer, house_canberra_act, canberra_act, box_house_canberra, matters_box_house, standing_matters_box, joint_standing_matters, standing_matters, vote*.

5.4 Word2Vec Clustering Experiment (Hierarchical + K-means)

This section clusters CBOW Word2Vec document embeddings with hierarchical clustering and K-means, then interprets clusters using token back-mapping.

What these concepts mean:
Word2Vec embeddings represent semantic context in dense vectors rather than sparse term counts.
K-means assigns documents to fixed cluster centroids, while **hierarchical clustering** reveals nested similarity structure.

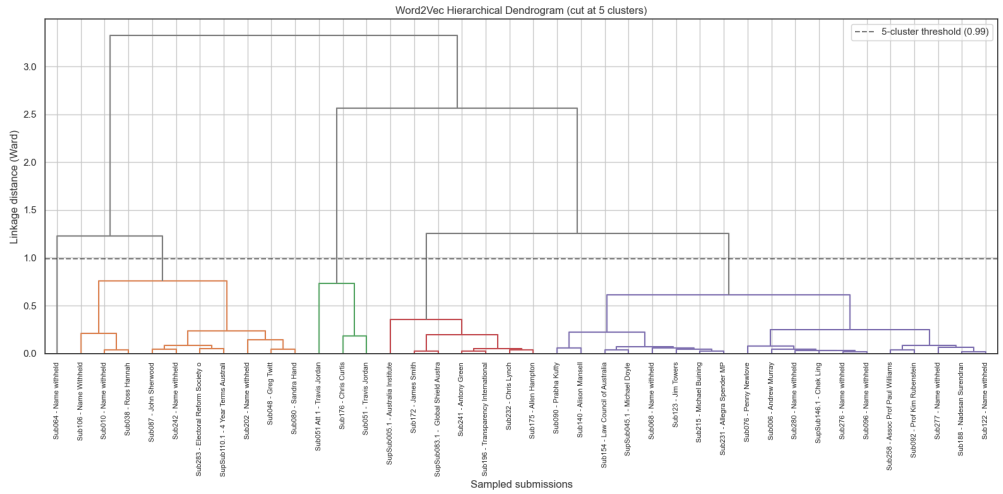
This complements TF-IDF by testing whether semantic embeddings produce coherent clusters beyond frequency-based weighting.

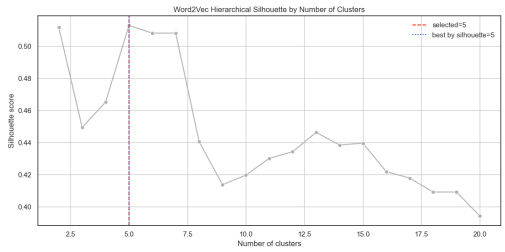
In [35]:

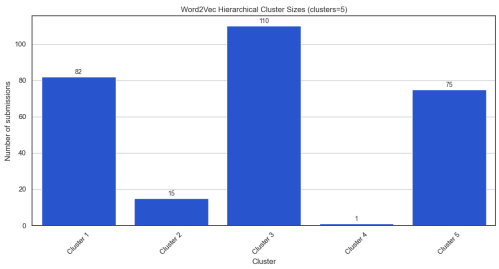
```
# 5.4A Word2Vec hierarchical clustering

X_w2v = X_cbow_w2v

w2v_hier_cluster_num_candidates = list(range(2, 21))
w2v_hier_eval = []
for cluster_num in w2v_hier_cluster_num_candidates:
    labels = AgglomerativeClustering(n_clusters=cluster_num,
linkage="ward").fit_predict(X_w2v)
    w2v_hier_eval.append(
        {
            "cluster_num": cluster_num,
            "silhouette_score": silhouette_score(X_w2v, labels),
        }
    )

w2v_hier_eval_df = pd.DataFrame(w2v_hier_eval)
w2v_hier_auto_best_cluster_num = int(

w2v_hier_eval_df.loc[w2v_hier_eval_df["silhouette_score"].idxmax(),
"cluster_num"]
)
w2v_hier_selected_cluster_num = 5

w2v_hier_labels = AgglomerativeClustering(
    n_clusters=w2v_hier_selected_cluster_num, linkage="ward"
).fit_predict(X_w2v)
corpus_df["w2v_hierarchical_cluster"] = w2v_hier_labels + 1

w2v_hier_counts = (
    corpus_df["w2v_hierarchical_cluster"]
    .value_counts()
    .sort_index()
    .rename_axis("cluster")
    .reset_index(name="count")
)
w2v_hier_counts["cluster"] =
w2v_hier_counts["cluster"].astype(str).radd("Cluster ")

w2v_hier_top_terms_tokens = top_tokens_from_cluster_assignments(
    lemmatized_docs,
    w2v_hier_labels,
    top_n=12,
)

LOGGER.debug(
    "Word2Vec hierarchical clustering complete: docs=%d,
clusters=%d",
    len(corpus_df),
    w2v_hier_selected_cluster_num,
)

# Dendrogram on a subsample for readability
```

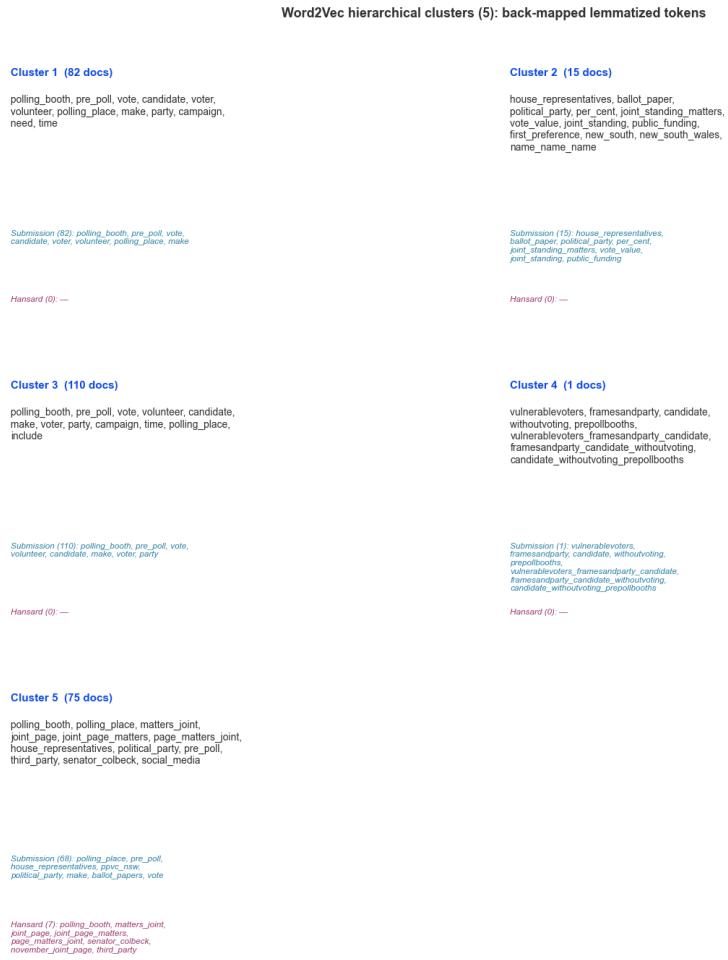
```
sample_size = min(40, X_w2v.shape[0])
sample_indices = np.random.RandomState(42).choice(X_w2v.shape[0],
size=sample_size, replace=False)
sample_features = X_w2v[sample_indices]
sample_labels = [Path(name).stem[:35] for name in
corpus_df.iloc[sample_indices]["file_name"]]

linkage_matrix = linkage(sample_features, method="ward")
threshold = (linkage_matrix[-w2v_hier_selected_cluster_num + 1, 2]
             + linkage_matrix[-w2v_hier_selected_cluster_num, 2]) /
2

plt.figure(figsize=(16, 8))
dendrogram(
    linkage_matrix,
    labels=sample_labels,
    leaf_rotation=90,
    leaf_font_size=8,
    color_threshold=threshold,
    above_threshold_color="grey",
)
plt.axhline(y=threshold, color="black", linestyle="--", alpha=0.5,
            label=f"{w2v_hier_selected_cluster_num}-cluster
threshold ({threshold:.2f})")
plt.title(f"Word2Vec Hierarchical Dendrogram (cut at
{w2v_hier_selected_cluster_num} clusters)")
plt.xlabel("Sampled submissions")
plt.ylabel("Linkage distance (Ward)")
plt.legend(loc="upper right")
plt.tight_layout()
plt.show()

# Silhouette curve and cluster sizes
fig, axes = plt.subplots(1, 2, figsize=(22, 6))

sns.lineplot(
    data=w2v_hier_eval_df, x="cluster_num", y="silhouette_score",
    marker="o", ax=axes[0], color=UTS_PALETTE["mid_grey"],
)
axes[0].axvline(w2v_hier_selected_cluster_num,
color=UTS_PALETTE["red"], linestyle="--",
                label=f"selected={w2v_hier_selected_cluster_num}")
axes[0].axvline(w2v_hier_auto_best_cluster_num,
color=UTS_PALETTE["blue"], linestyle=":",
                label=f"best by silhouette=
{w2v_hier_auto_best_cluster_num}")
axes[0].set_title("Word2Vec Hierarchical Silhouette by Number of
Clusters")
axes[0].set_xlabel("Number of clusters")
axes[0].set_ylabel("Silhouette score")
axes[0].legend(frameon=False)

sns.barplot(data=w2v_hier_counts, x="cluster", y="count",
```

```
color=UTS_PALETTE["blue"], ax=axes[1])
axes[1].set_title(f"Word2Vec Hierarchical Cluster Sizes (clusters=
{w2v_hier_selected_cluster_num})")
axes[1].set_xlabel("Cluster")
axes[1].set_ylabel("Number of submissions")
for container in axes[1].containers:
    axes[1].bar_label(container, padding=2, fontsize=10)
axes[1].tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

w2v_hier_source_bkdn = source_breakdown_tokens(
    lemmatized_docs, w2v_hier_labels,
    corpus_df["evidence_style"].values, top_n=8,
)
plot_cluster_term_cards(
    f"Word2Vec hierarchical clusters
({w2v_hier_selected_cluster_num}): back-mapped lemmatized tokens",
    w2v_hier_top_terms_tokens,
    source_breakdown=w2v_hier_source_bkdn,
)

print_cluster_summary(
    f"Word2Vec hierarchical clusters (selected=
{w2v_hier_selected_cluster_num}, "
    f"auto-best by silhouette={w2v_hier_auto_best_cluster_num}):",
    w2v_hier_top_terms_tokens,
)
```










```
Word2Vec hierarchical clusters (selected=5, auto-best by
silhouette=5):
- Cluster 1 (82 docs)
  polling_booth, pre_poll, vote, candidate, voter, volunteer,
polling_place, make, party, campaign, need, time
- Cluster 2 (15 docs)
  house_representatives, ballot_paper, political_party,
per_cent, joint_standing_matters, vote_value, joint_standing,
public_funding, first_preference, new_south, new_south_wales,
name_name_name
- Cluster 3 (110 docs)
  polling_booth, pre_poll, vote, volunteer, candidate, make,
voter, party, campaign, time, polling_place, include
- Cluster 4 (1 docs)
  vulnerablevoters, framesandparty, candidate, withoutvoting,
prepollbooths, vulnerablevoters_framesandparty_candidate,
framesandparty_candidate_withoutvoting,
candidate_withoutvoting_prepollbooths
- Cluster 5 (75 docs)
  polling_booth, polling_place, matters_joint, joint_page,
joint_page_matters, page_matters_joint, house_representatives,
political_party, pre_poll, third_party, senator_colbeck,
social_media
```

The silhouette curve peaks at num of clusters = 5 (≈ 0.54). Notably, the Word2Vec silhouette scores are **substantially higher** than those observed in the TF-IDF experiments (≈ 0.08–0.22), confirming that dense CBOW embeddings produce tighter, better separated document clusters than sparse frequency based representations. Beyond num of clusters = 6 the curve drops sharply to ≈ 0.43 and remains flat through k = 20.

- **Cluster 1 (82 docs) — Polling operations and voter participation.** Top terms: *polling_booth, pre_poll, vote, candidate, voter, volunteer, polling_place, make, party, campaign, need, time*.

- **Cluster 2 (15 docs) — Institutional electoral framework and funding.** Top terms: *house_representatives, ballot_paper, political_party, per_cent, joint_standing_matters, vote_value, joint_standing, public_funding, first_preference, new_south, new_south_wales, name_name_name*. All 15 documents are written submissions. The vocabulary is distinctly institutional *house_representatives*, *political_party*, *public_funding*, *vote_value* indicating formal policy proposals around parliamentary representation, funding mechanisms, and preferential voting. The term *new_south_wales* suggests a geographic focus on NSW electoral matters.

- **Cluster 3 (110 docs) — General election discourse.** Top terms: *polling_booth, pre_poll, vote, volunteer, candidate, make, voter, party, campaign, time, polling_place, include*. The largest cluster overlaps heavily with Cluster 1 in its top vocabulary, differing mainly in rank order (*volunteer* rises and *need* is replaced by *include*).

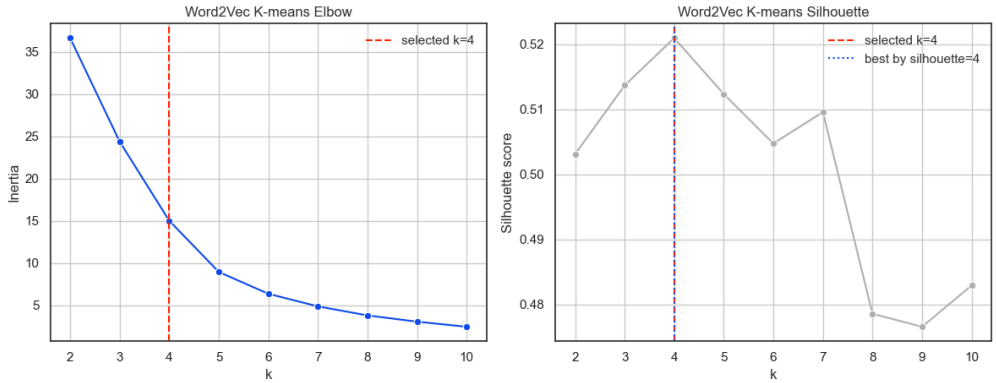
- **Cluster 4 (1 doc) — Outlier with corrupted tokens.** Top terms: *vulnerablevoters, framesandparty, candidate, withoutvoting, prepollbooths, vulnerablevoters_framesandparty_candidate, framesandparty_candidate_withoutvoting, candidate_withoutvoting_prepollbooths*. This singleton contains concatenated, malformed tokens (e.g., *vulnerablevoters*, *framesandparty*, *withoutvoting*).

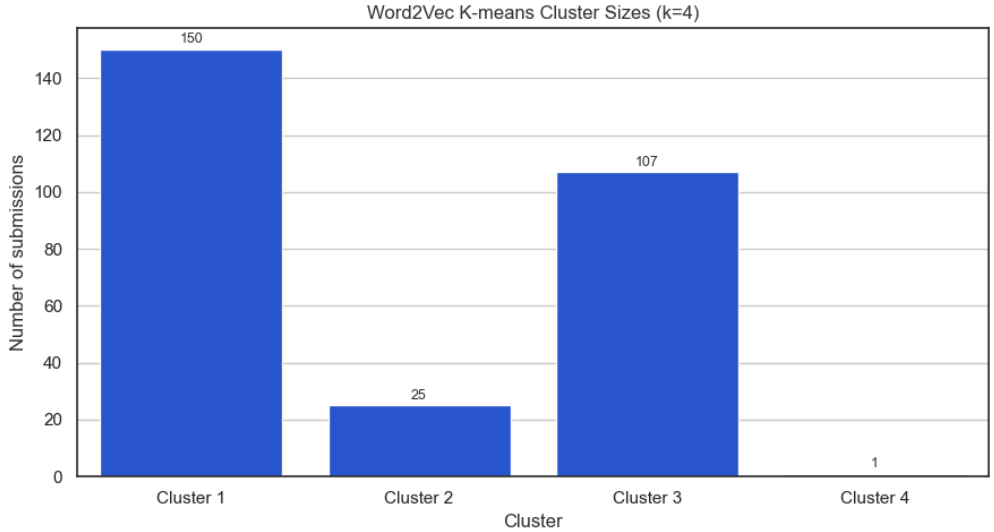
- **Cluster 5 (75 docs) — Parliamentary proceedings and mixed policy themes.** Top terms: *polling_booth, polling_place, matters_joint, joint_page, joint_page_matters, page_matters_joint, house_representatives, political_party, pre_poll, third_party, senator_colbeck, social_media*. This is the **only cluster containing Hansard transcripts** (7 out of 75 documents).

In [36]:

```
# 5.4B Word2Vec K-means clustering

# Evaluate candidate k values up to 10, but force selected value to
k=4
w2v_k_candidates = list(range(2, 11))
w2v_kmeans_eval = []
w2v_kmeans_label_map = {}

for k in w2v_k_candidates:
    model = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = model.fit_predict(X_w2v)
    w2v_kmeans_label_map[k] = labels
    w2v_kmeans_eval.append(
        {
            "k": k,
            "inertia": model.inertia_,
            "silhouette_score": silhouette_score(X_w2v, labels),
        }
    )

w2v_kmeans_eval_df = pd.DataFrame(w2v_kmeans_eval)
w2v_auto_best_k =
int(w2v_kmeans_eval_df.loc[w2v_kmeans_eval_df["silhouette_score"].id
xmax(), "k"])
w2v_best_k = 4

if w2v_best_k not in w2v_kmeans_label_map:
    raise ValueError("Selected w2v_best_k is not in evaluated
w2v_k_candidates")

w2v_kmeans_labels = w2v_kmeans_label_map[w2v_best_k]
corpus_df["w2v_kmeans_cluster"] = w2v_kmeans_labels + 1

w2v_kmeans_counts = (
    corpus_df["w2v_kmeans_cluster"]
    .value_counts()
    .sort_index()
    .rename_axis("cluster")
    .reset_index(name="count")
)
w2v_kmeans_counts["cluster"] =
w2v_kmeans_counts["cluster"].astype(str).radd("Cluster ")

# Backward token mapping for Word2Vec K-means clusters
w2v_kmeans_top_terms_tokens = top_tokens_from_cluster_assignments(
    lemmatized_docs,
    w2v_kmeans_labels,
    top_n=12,
)

# Plot K-means evaluation metrics
fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
sns.lineplot(data=w2v_kmeans_eval_df, x="k", y="inertia",
```

```
marker="o", ax=axes[0], color=UTS_PALETTE["blue"])
axes[0].axvline(w2v_best_k, color=UTS_PALETTE["red"], linestyle="--
", label="selected k=4")
axes[0].set_title("Word2Vec K-means Elbow")
axes[0].set_xlabel("k")
axes[0].set_ylabel("Inertia")
axes[0].legend(frameon=False)

sns.lineplot(data=w2v_kmeans_eval_df, x="k", y="silhouette_score",
marker="o", ax=axes[1], color=UTS_PALETTE["mid_grey"])
axes[1].axvline(w2v_best_k, color=UTS_PALETTE["red"], linestyle="--
", label="selected k=4")
axes[1].axvline(w2v_auto_best_k, color=UTS_PALETTE["blue"],
linestyle=":", label=f"best by silhouette={w2v_auto_best_k}")
axes[1].set_title("Word2Vec K-means Silhouette")
axes[1].set_xlabel("k")
axes[1].set_ylabel("Silhouette score")
axes[1].legend(frameon=False)
plt.tight_layout()
plt.show()

# Cluster size barplot
fig, ax = plt.subplots(figsize=(9, 5))
sns.barplot(data=w2v_kmeans_counts, x="cluster", y="count",
color=UTS_PALETTE["blue"], ax=ax)
ax.set_title("Word2Vec K-means Cluster Sizes (k=4)")
ax.set_xlabel("Cluster")
ax.set_ylabel("Number of submissions")
for container in ax.containers:
    ax.bar_label(container, padding=2, fontsize=9)
plt.tight_layout()
plt.show()

if "plot_cluster_term_cards" in globals():
    w2v_kmeans_source_bkdn = source_breakdown_tokens(
        lemmatized_docs, w2v_kmeans_labels,
        corpus_df["evidence_style"].values, top_n=8,
    )
    plot_cluster_term_cards(
        "Word2Vec K-means clusters: back-mapped lemmatized tokens",
        w2v_kmeans_top_terms_tokens,
        source_breakdown=w2v_kmeans_source_bkdn,
    )

print(f"\nWord2Vec K-means selected k: {w2v_best_k} (auto-best by
silhouette was {w2v_auto_best_k})")
print_cluster_summary("Word2Vec K-means clusters (back-mapped
lemmatized tokens):", w2v_kmeans_top_terms_tokens)
```





Word2Vec K-means clusters: back-mapped lemmatized tokens

Cluster 1 (150 docs)

polling_booth, polling_place, pre_poll, house_representatives, matters_joint, joint_page, joint_page_matters, page_matters_joint, vote, make, political_party, third_party

Submission (144): pre_poll, polling_booth, polling_place, vote, house_representatives, make, include, voter

Hansard (6): polling_booth, matters_joint, joint_page, joint_page_matters, page_matters_joint, november_joint_page, senator_colbeck, third_party

Cluster 2 (25 docs)

house_representatives, ballot_paper, ppvc_nsw, political_party, social_media, per_cent, vote_value, joint_standing_matters, joint_standing, state_division, ppvc_total, ppvc_total_votes

Submission (24): house_representatives, ballot_paper, ppvc_nsw, political_party, per_cent, vote_value, joint_standing_matters, state_division

Hansard (1): friday_october, foreign_interference, joint_page, matters_joint, friday_october_joint, october_joint_page, joint_page_matters, page_matters_joint

Cluster 3 (107 docs)

polling_booth, candidate, vote, pre_poll, volunteer, voter, polling_place, make, party, campaign, time, need

Submission (107): polling_booth, candidate, vote, pre_poll, volunteer, voter, polling_place, make

Hansard (0): —

Cluster 4 (1 docs)

vulnerablevoters, framesandparty, candidate, withoutvoting, prepollbooths, vulnerablevoters_framesandparty_candidate, framesandparty_candidate_withoutvoting, candidate_withoutvoting_prepollbooths

Submission (1): vulnerablevoters, framesandparty, candidate, withoutvoting, prepollbooths, vulnerablevoters_framesandparty_candidate, framesandparty_candidate_withoutvoting, candidate_withoutvoting_prepollbooths

Hansard (0): —

```
Word2Vec K-means selected k: 4 (auto-best by silhouette was 4)

Word2Vec K-means clusters (back-mapped lemmatized tokens):
- Cluster 1 (150 docs)
  polling_booth, polling_place, pre_poll,
```

```
house_representatives, matters_joint, joint_page,
joint_page_matters, page_matters_joint, vote, make,
political_party, third_party
- Cluster 2 (25 docs)
  house_representatives, ballot_paper, ppvc_nsw,
political_party, social_media, per_cent, vote_value,
joint_standing_matters, joint_standing, state_division,
ppvc_total, ppvc_total_votes
- Cluster 3 (107 docs)
  polling_booth, candidate, vote, pre_poll, volunteer, voter,
polling_place, make, party, campaign, time, need
- Cluster 4 (1 docs)
  vulnerablevoters, framesandparty, candidate, withoutvoting,
prepollbooths, vulnerablevoters_framesandparty_candidate,
framesandparty_candidate_withoutvoting,
candidate_withoutvoting_prepollbooths
```

The elbow plot shows a pronounced bend at k = 4: inertia falls steeply from ≈ 36 (k = 2) to ≈ 15 (k = 4), then flattens into a gentle decline through k = 10, confirming that the fourth centroid captures the last substantial chunk of within cluster variance. The silhouette plot reinforces this choice the silhouette score peaks at k = 4 (≈ 0.52), the highest across all candidates.

Comparison with hierarchical clustering. The hierarchical experiment used num of clusters = 5 and achieved a peak silhouette of ≈ 0.54. K-means at k = 4 reaches ≈ 0.52, a marginally lower score but with one fewer cluster, resulting in a simpler partition that sacrifices very little cohesion. The main structural difference is that the hierarchical five cluster solution splits the large polling/proceedings group into two sub clusters, whereas K-means folds them into a single large Cluster 1 (150 docs).

- **Cluster 1 (150 docs) Polling logistics and parliamentary proceedings.** Top terms: *polling_booth, polling_place, pre_poll, house_representatives, matters_joint, joint_page, vote, make, political_party, third_party*. This is the largest cluster and the only one containing Hansard transcripts (6 of 150 docs). The vocabulary blends two registers: written submissions focused on physical polling infrastructure (*polling_booth*, *polling_place*, *pre_poll*) and parliamentary committee language (*matters_joint*, *joint_page*, *house_representatives*). This merging suggests that, in the Word2Vec embedding space, the procedural language of Hansard is semantically closer to polling logistics submissions than to institutional policy text.

- **Cluster 2 (25 docs) Institutional policy, electoral statistics, and media regulation.** Top terms: *house_representatives, ballot_paper, ppvc_nsw, political_party, social_media, per_cent, vote_value, joint_standing_matters, state_division, ppvc_total_votes*. Almost exclusively written submissions (24 of 25), this cluster stands out for its quantitative and regulatory vocabulary — *per_cent*, *vote_value*, *ppvc_total*, *state_division* — pointing to documents that discuss electoral formulas, proportional representation, and state level vote distributions. The presence of *social_media* and *ppvc_nsw* further indicates submissions addressing media regulation and NSW specific Pre Poll Voting Centre data. The single Hansard document in this cluster uses committee calendar tokens (*friday*, *october*), suggesting it was pulled in by scheduling metadata rather than policy content.

- **Cluster 3 (107 docs) Grassroots election participation and campaigning.** Top terms: *polling_booth, candidate, vote, pre_poll, volunteer, voter, polling_place, make, party, campaign, time, need*. Entirely composed of written submissions (107 of 107), with zero Hansard representation. The distinguishing terms relative to Cluster 1 are *volunteer*, *candidate*, *campaign*, *time*, and *need* — action oriented, grassroots vocabulary that foregrounds the human experience of volunteering at booths, running as a candidate, and managing campaign logistics.

- **Cluster 4 (1 doc) Outlier with concatenated tokens.** Top terms: *vulnerablevoters, framesandparty, withoutvoting, prepollbooths*. A singleton submission whose text was

extracted without proper whitespace, producing fused multi word tokens. The malformed vocabulary has no overlap with any other cluster, causing K-means to isolate it as an extreme outlier.

6. Conclusion

Answering the research questions

1. What topics and issues appear most frequently across submissions?

The corpus is anchored by a consistent core vocabulary centred on the Australian Electoral Commission (*aec*), democratic participation (*vote*, *voters*, *voting*, *party*, *political*, *people*), and the physical infrastructure of elections (*polling_place*, *polling_booth*, *pre_poll*, *ballot_paper*). NPMI phrase detection elevated multi word policy concepts — *fixed_term*, *political_advertising*, *truth_political_advertising*, *social_media*, *postal_vote*, *house representatives*, *joint_standing* — that single token frequency charts would have missed. LDA topic modeling confirmed these thematic clusters at a latent level, grouping terms into coherent topics around electoral administration, parliamentary procedure, campaign regulation, and voter access. Together, the evidence shows that the inquiry discourse is dominated by the practical mechanics of how elections are run, with secondary but persistent attention to advertising regulation and media influence.

2. What are the most frequent inquiries raised by the submissions and the Hansard transcripts?

Written submissions are dominated by the practical mechanics of elections: *polling_booth*, *polling_place*, *pre_poll*, *vote*, *candidate*, and *volunteer* rank highest, revealing that the public's primary concerns centre on polling logistics, ballot access, and the volunteer workforce at voting centres. Hansard transcripts foreground procedural and deliberative language *people*, *chair*, *senator*, *joint_standing*, *committee* reflecting the structured question and answer format of live parliamentary hearings.

3. Is there any common topic covered by the submissions and the Hansard transcripts?

Yes. Despite their different registers formal written submissions versus conversational oral testimony — both evidence streams share a broad thematic core around the AEC, polling logistics, political parties, and voter participation. Terms like *aec*, *polling*, *party*,

voters, and *political* appear prominently in both the overall corpus chart and the Hansard specific chart. The word cloud of the fully preprocessed corpus confirms this overlap visually, with *polling_place*, *polling_booth*, *ballot_paper*, and *vote* dominating regardless of evidence style. Where the two streams diverge is in register and emphasis: Hansard transcripts carry more procedural vocabulary (*chair*, *senator*, *joint*, *terms*) and conversational markers (*know*, *going*, *time*), while written submissions are denser in policy specific phrases (*fixed_term*, *political_advertising*, *postal_vote*). The shared core confirms that both streams address the same inquiry, while the divergences reflect the complementary roles of written evidence and oral examination in parliamentary processes.

Practical implications

- **For the JSCEM and policymakers:** The NLP analysis surfaces the dominant public concerns without requiring manual reading of all 299 submissions. Polling place logistics, volunteer management, ballot paper design, and political advertising regulation emerge as the issues with the broadest constituency across submitter types.
- **For future inquiry analysis:** The modular pipeline (ingestion, preprocessing, phrase detection, modeling) is reusable for other parliamentary inquiries. The Hansard specific stopword approach demonstrates that evidence stream aware preprocessing can improve interpretability without distorting the underlying data.

Limitations

- **Corpus scale.** With 306 documents the corpus is modest; larger inquiries would benefit from the same pipeline but may require more aggressive dimensionality reduction or sampling strategies.
- **PDF extraction noise.** Some submissions contained scanned images, headers/footers, and formatting artifacts that survived text extraction. While preprocessing removed much of this noise, a small residual effect on token counts is likely.
- **Submitter type inference.** The `submitter_type` field was inferred heuristically from filenames and metadata rather than from an authoritative registry, which means some classifications (especially "Unknown") may be imprecise.
- **Topic model sensitivity.** LDA coherence guided model selection, but coherence is an approximation of human interpretability, not a guarantee. Alternative topic counts or methods (e.g., BERTopic) might surface different thematic decompositions.

Recommended next steps

1. **Sentiment and stance analysis** extend the pipeline with sentiment scoring or stance detection to measure not just what submitters discuss but how they feel about it (e.g., support vs. opposition to fixed term parliaments).
2. **Cross inquiry comparison** apply the same pipeline to previous JSCEM inquiries (e.g., 2022 federal election) to track how public concerns evolve across electoral cycles.
3. **Transformer based topic modeling** experiment with BERTopic or similar embedding based approaches to test whether contextual semantics produce more granular or interpretable topics.

7. References

Australian Parliament House. (n.d.). *Making a submission to a committee inquiry*. Retrieved March 9, 2026, from https://www.aph.gov.au/Parliamentary_Business/Committees/House/Making_a_submiss
Australian Parliament House. (n.d.). *Inquiry into the 2025 federal election: Submissions*. Retrieved March 9, 2026, from https://www.aph.gov.au/Parliamentary_Business/Committees/Joint/Electoral_Matters/20
Australian Electoral Commission. (2025, August 11). *Frequently asked questions*. Retrieved March 9, 2026, from https://www.aec.gov.au/faqs/

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.